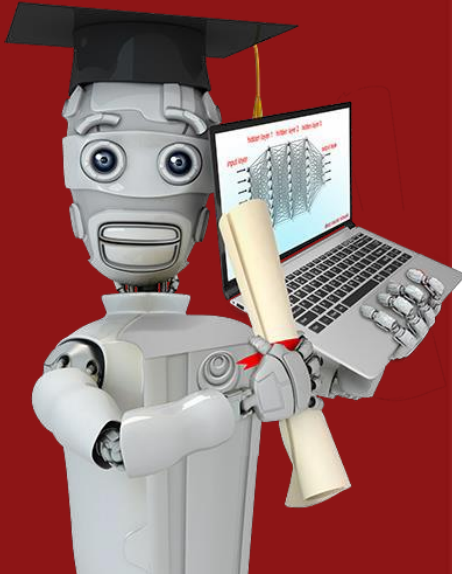




Stanford
ONLINE



Introduction to Neural Networks

Andrew Ng

Neural networks

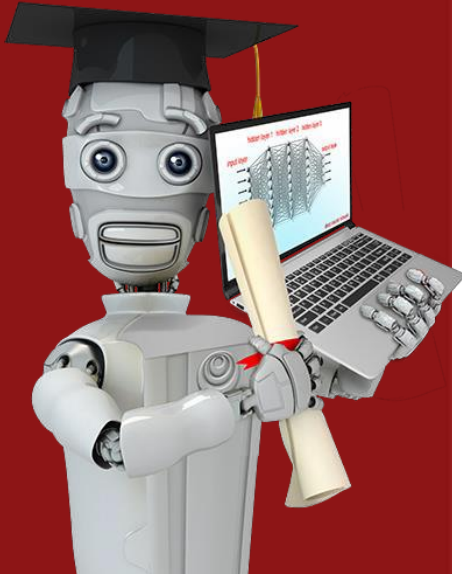
Origins: Algorithms that try to mimic the brain.



Used in the 1980's and early 1990's.
Fell out of favor in the late 1990's.

Resurgence from around 2005.

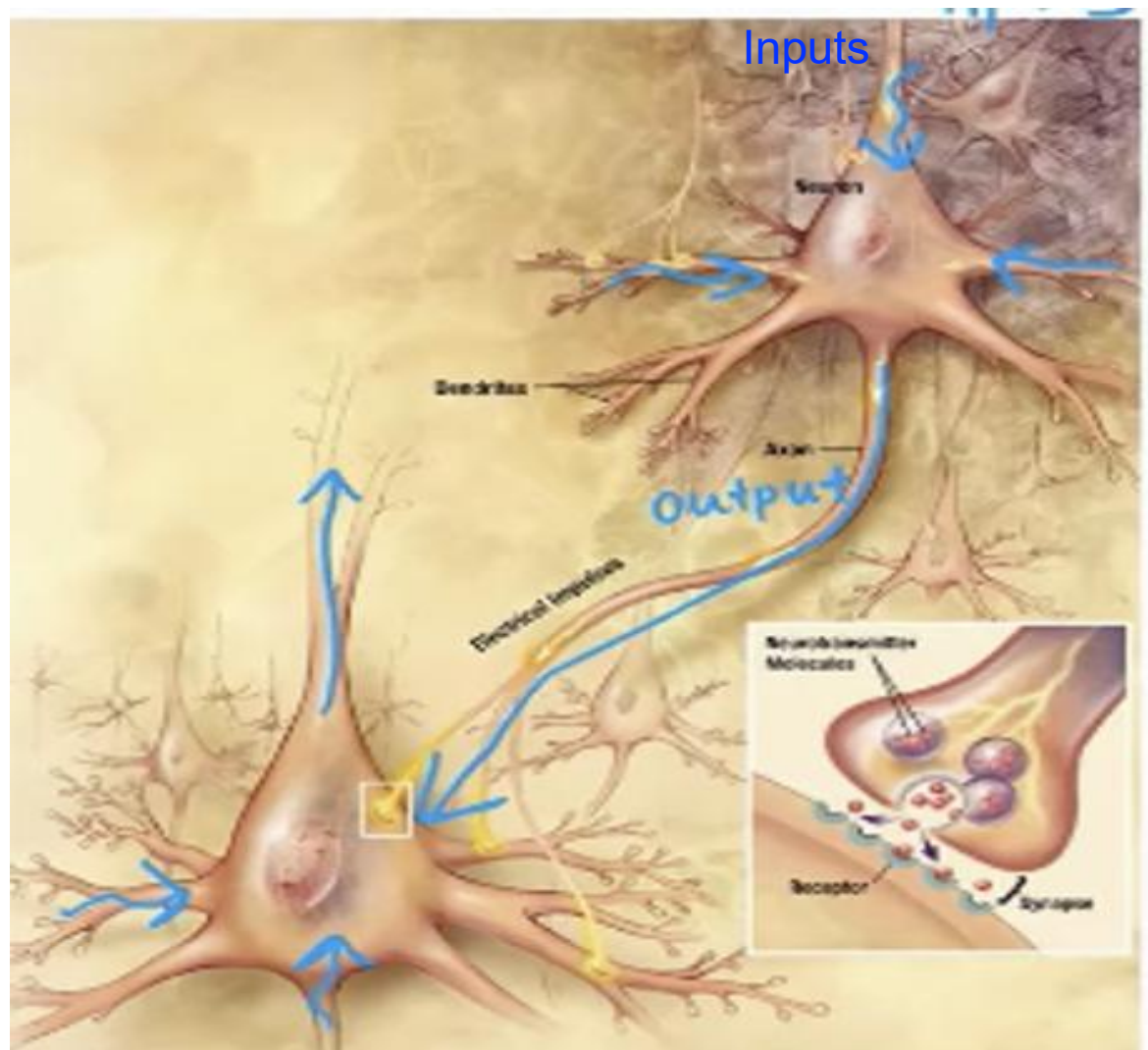
speech → images → text (NLP) → ...



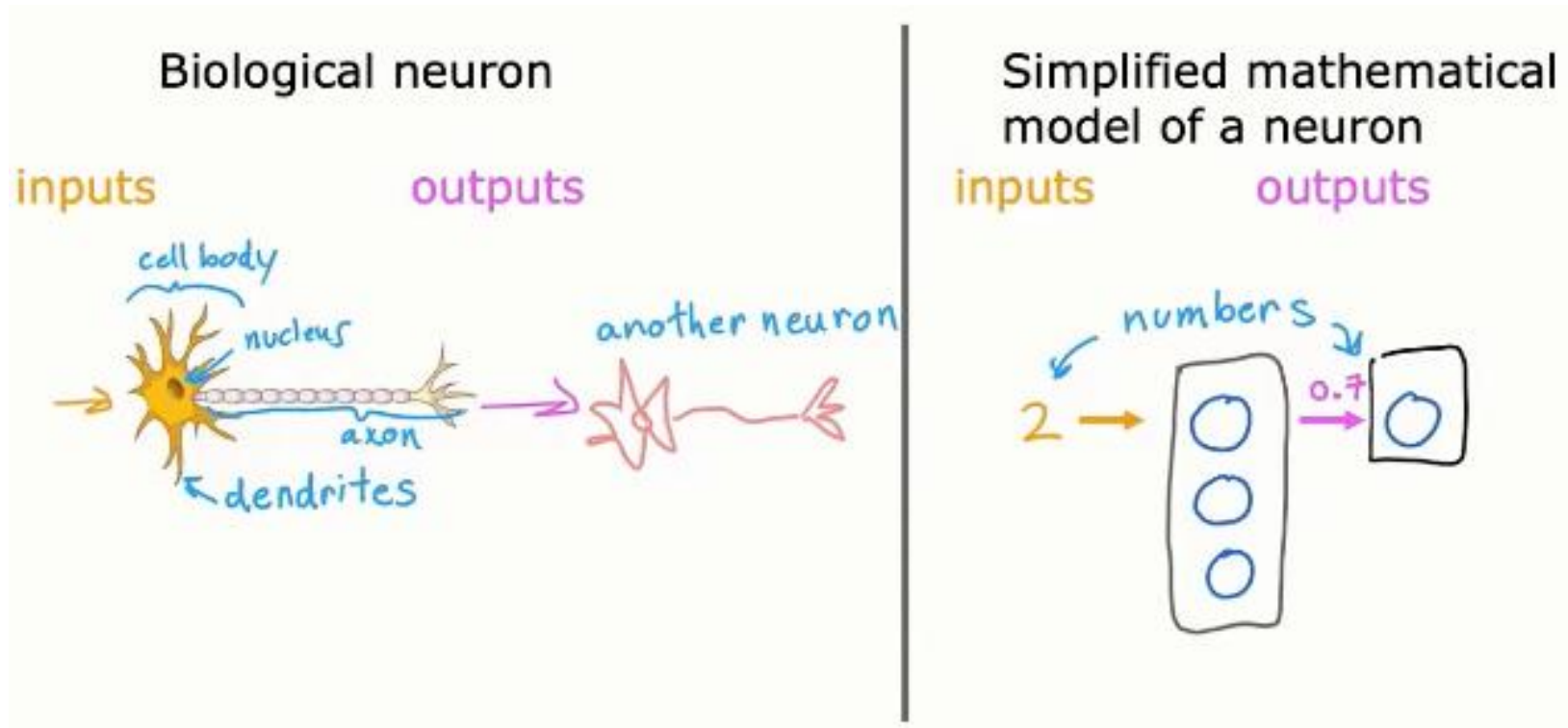
Neural Network Intuition

Neurons and the Brain

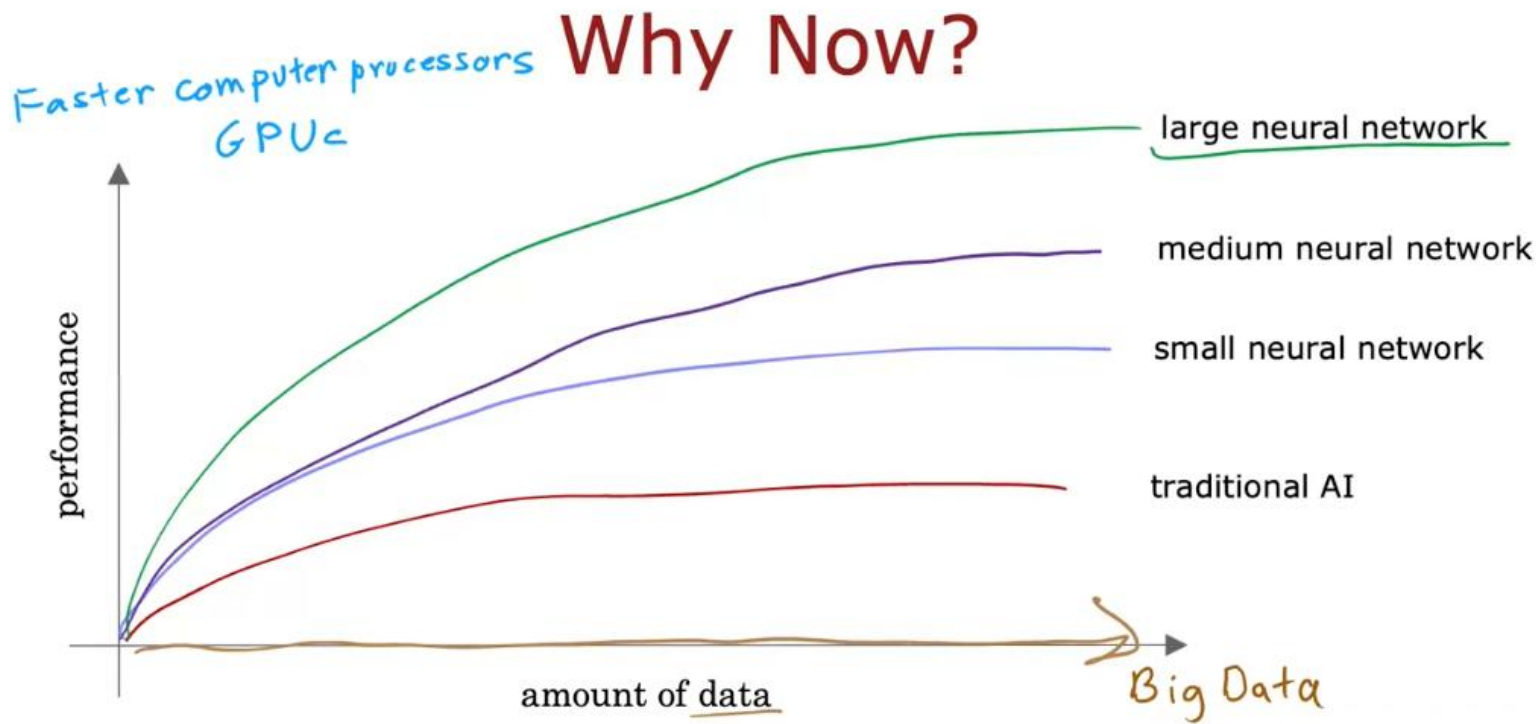
Biological Neuron: How the Brain Processes Information



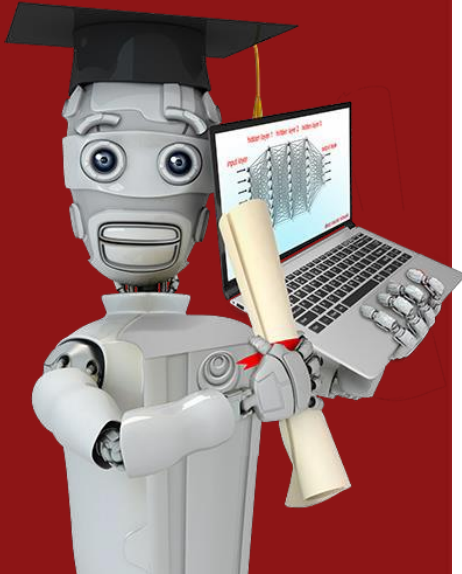
Biological vs. Artificial Neurons



Like biological neurons, artificial neurons work to process input data and Generates output, which is fed to the next neurons



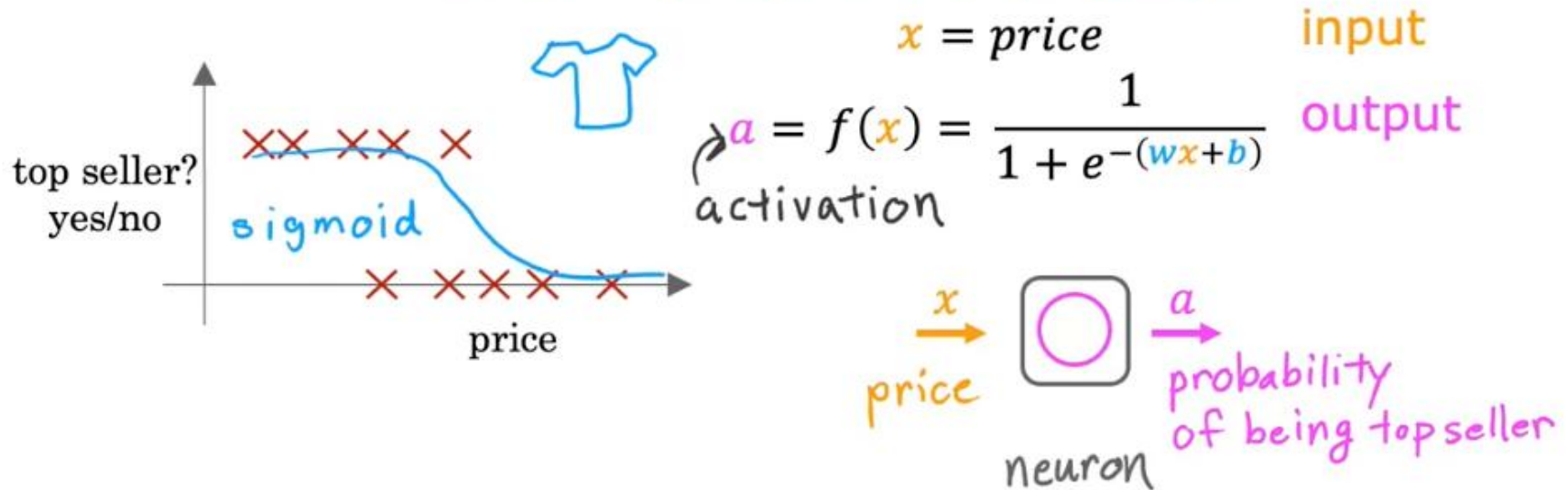
Neural networks are the mechanism that is able to cope with the increasing amount of data and the faster computing powers



Neural Network Intuition

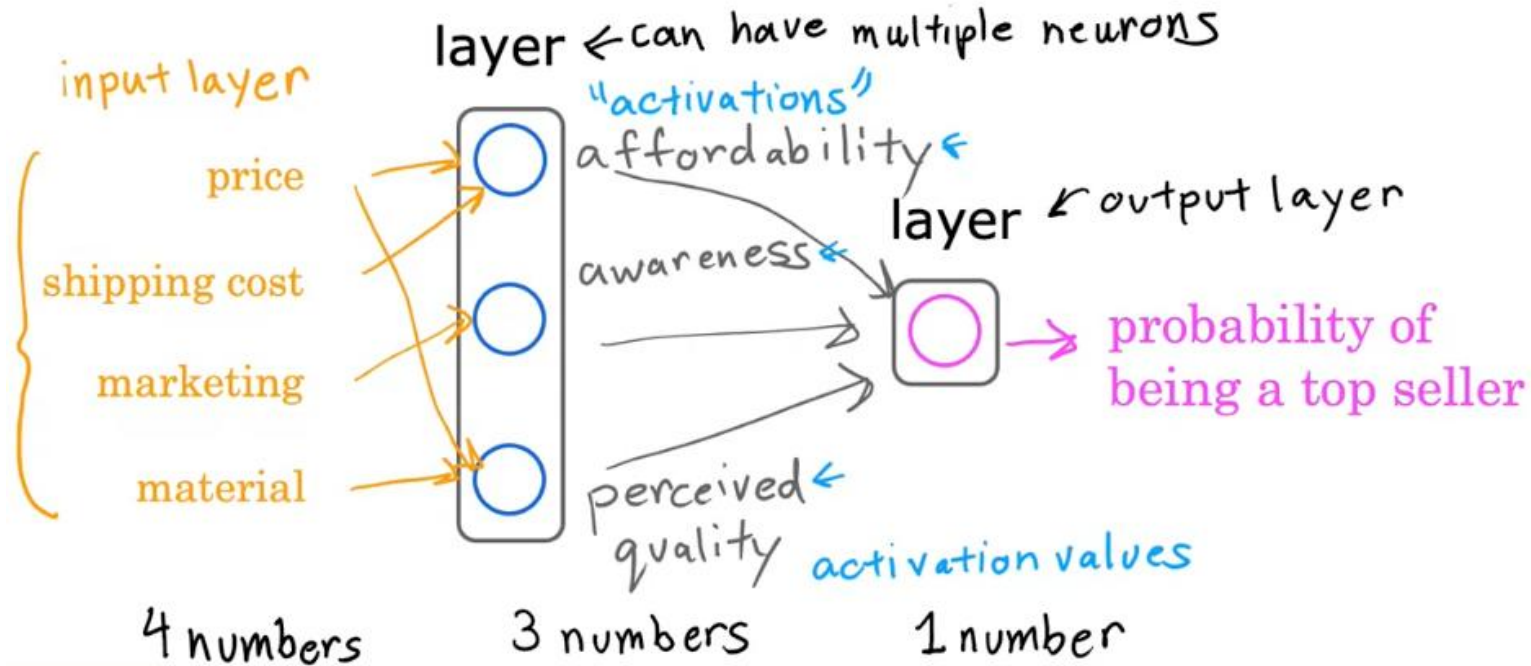
Demand Prediction

Demand Prediction



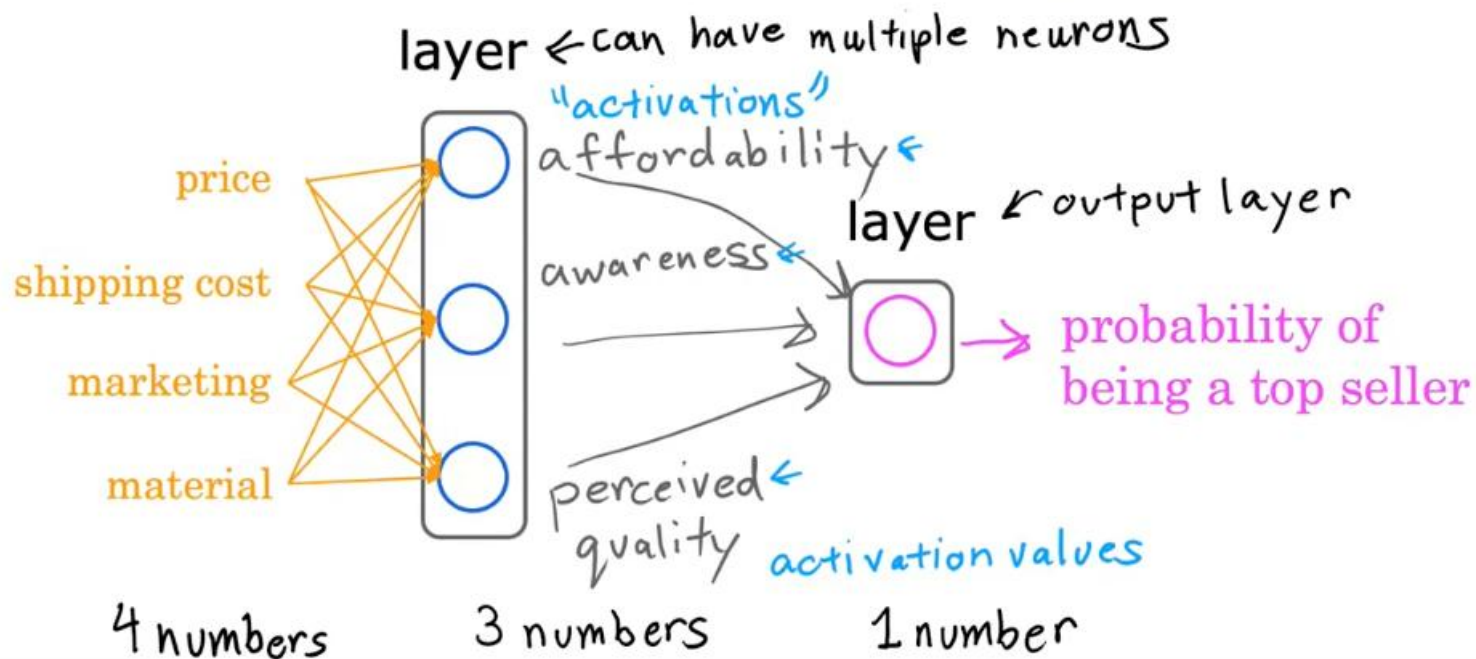
The sigmoid function which is used to predict whether a T-shirt is top Seller or Not (classification) is the work of a single Neurin on Neural Networks

Demand Prediction



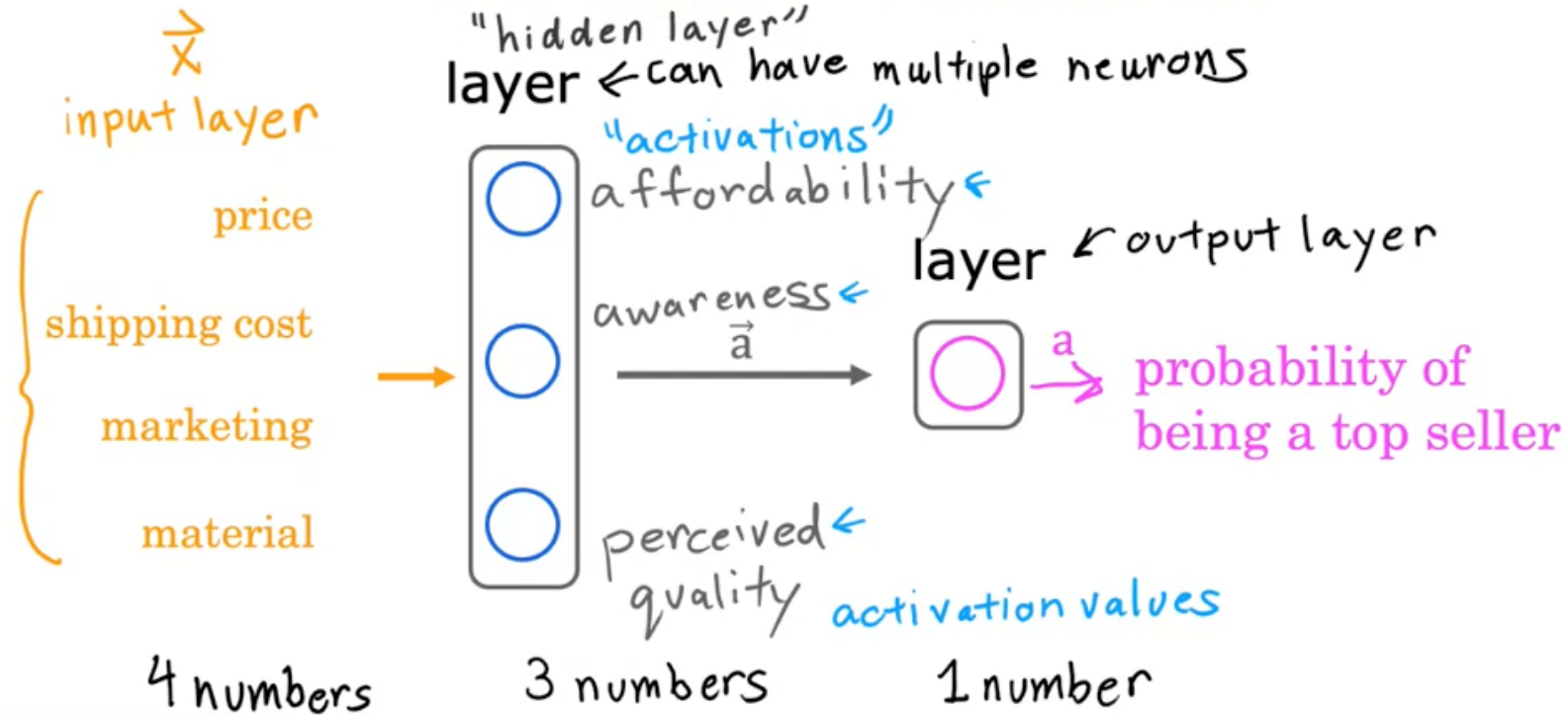
Here each neuron value like affordability is decided by the input features, in this case Price and shipping cost. The outcome of this neuron is called Activation

Demand Prediction



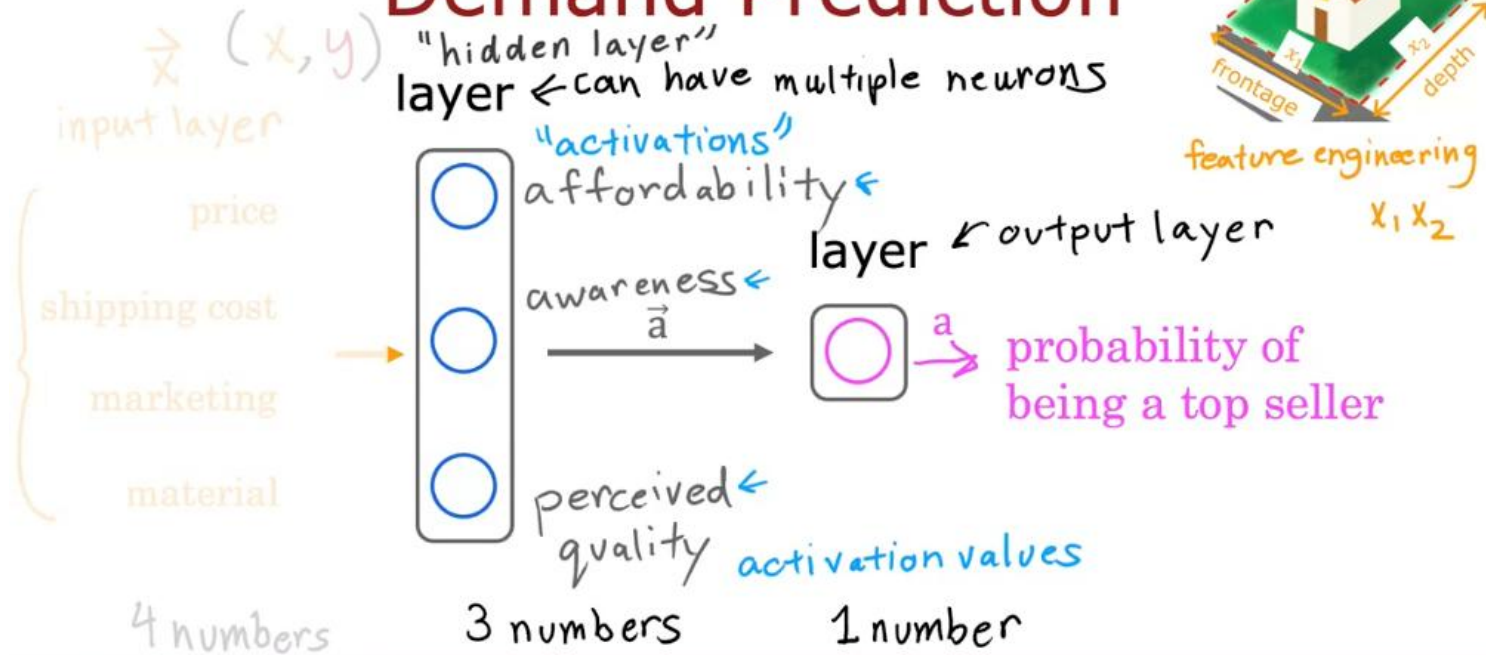
Instead of deciding the features that affect each node in the hidden layer, we input All features to each node and let the nodes decide which features affect them more

Demand Prediction



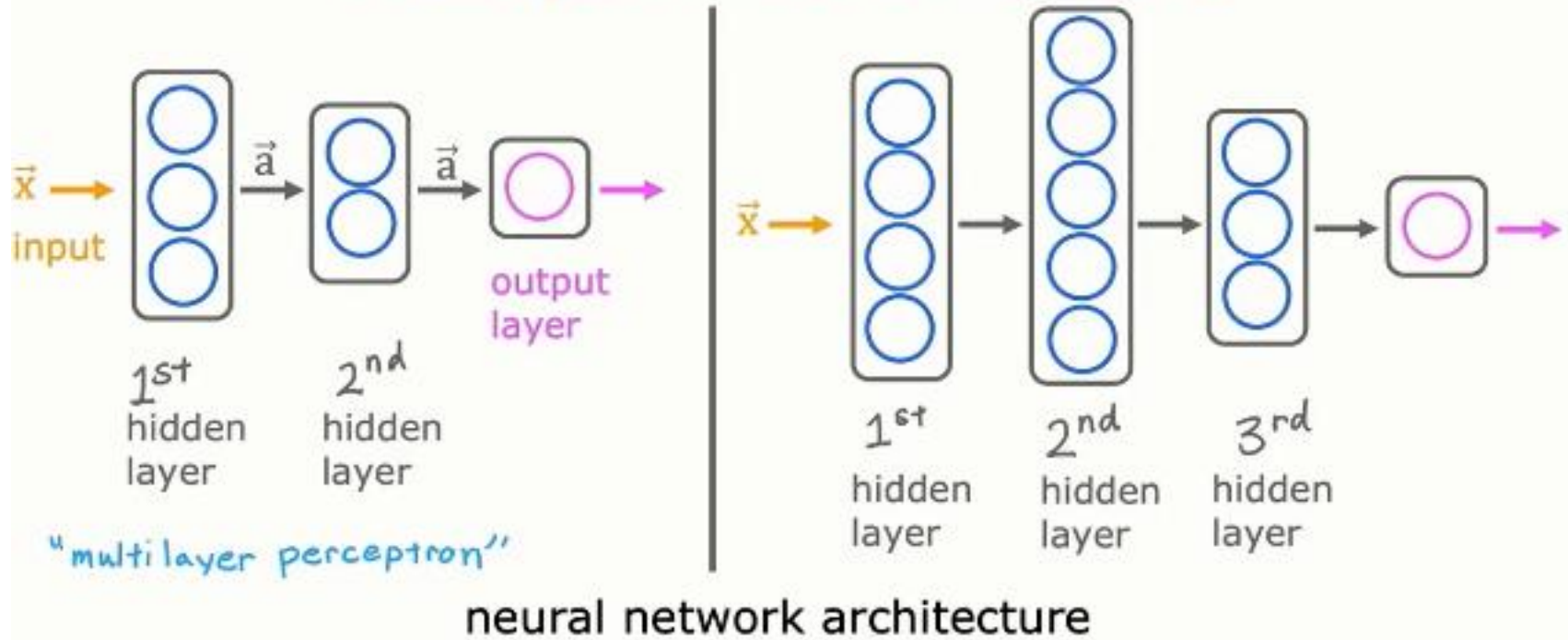
The values of the input layer, the hidden Layer are treated as vectors, while the output is a single value

Demand Prediction

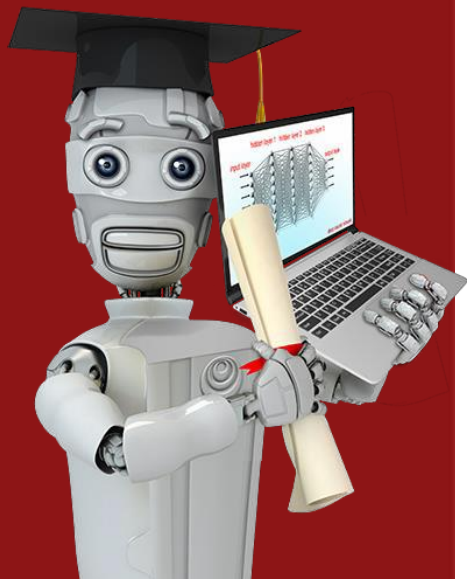


The hidden layer works as one type of Feature Engineering

Multiple hidden layers



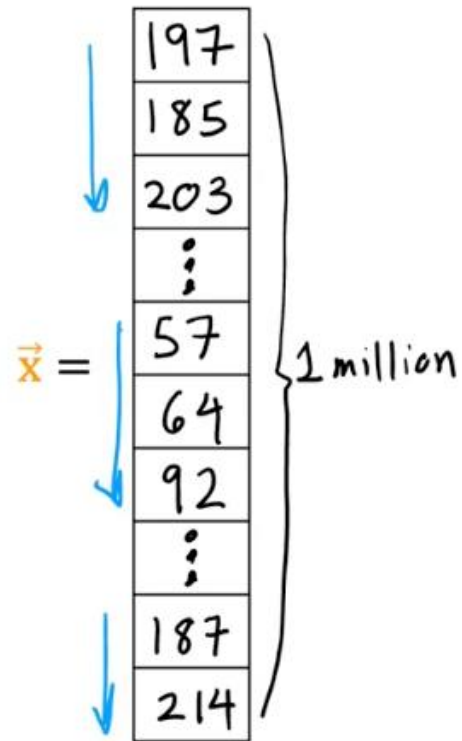
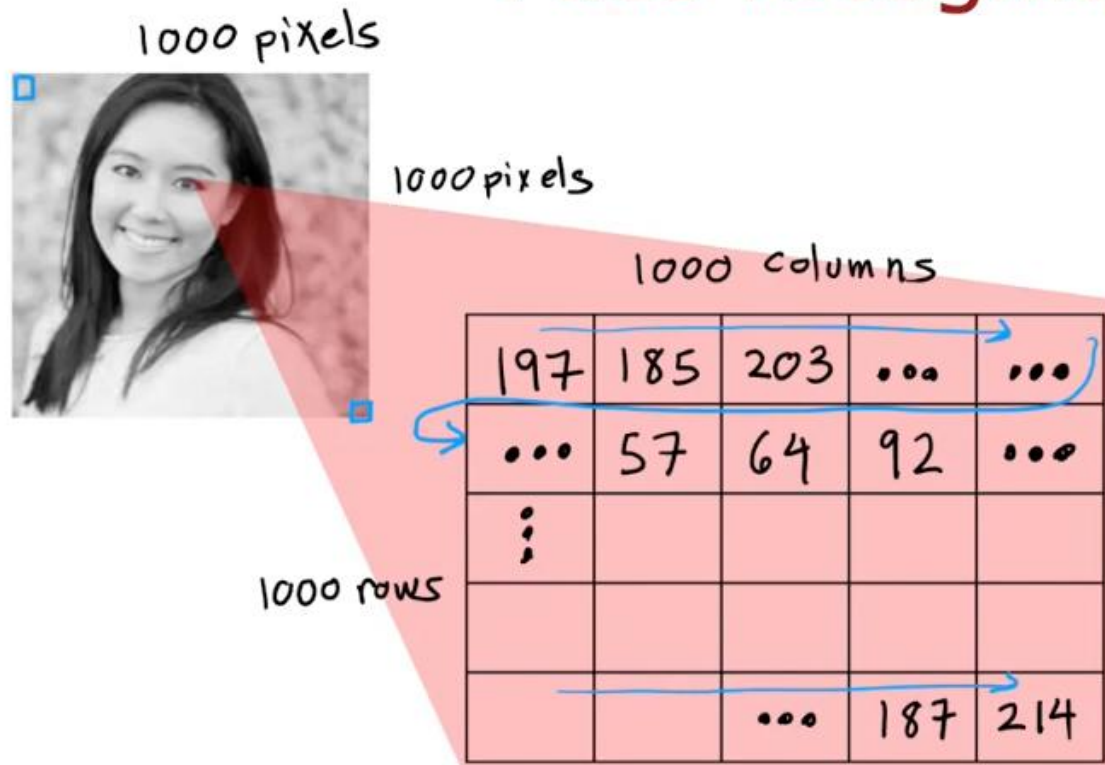
Neural Networks might have more neurons in a hidden layer, and more than one Hidden layer



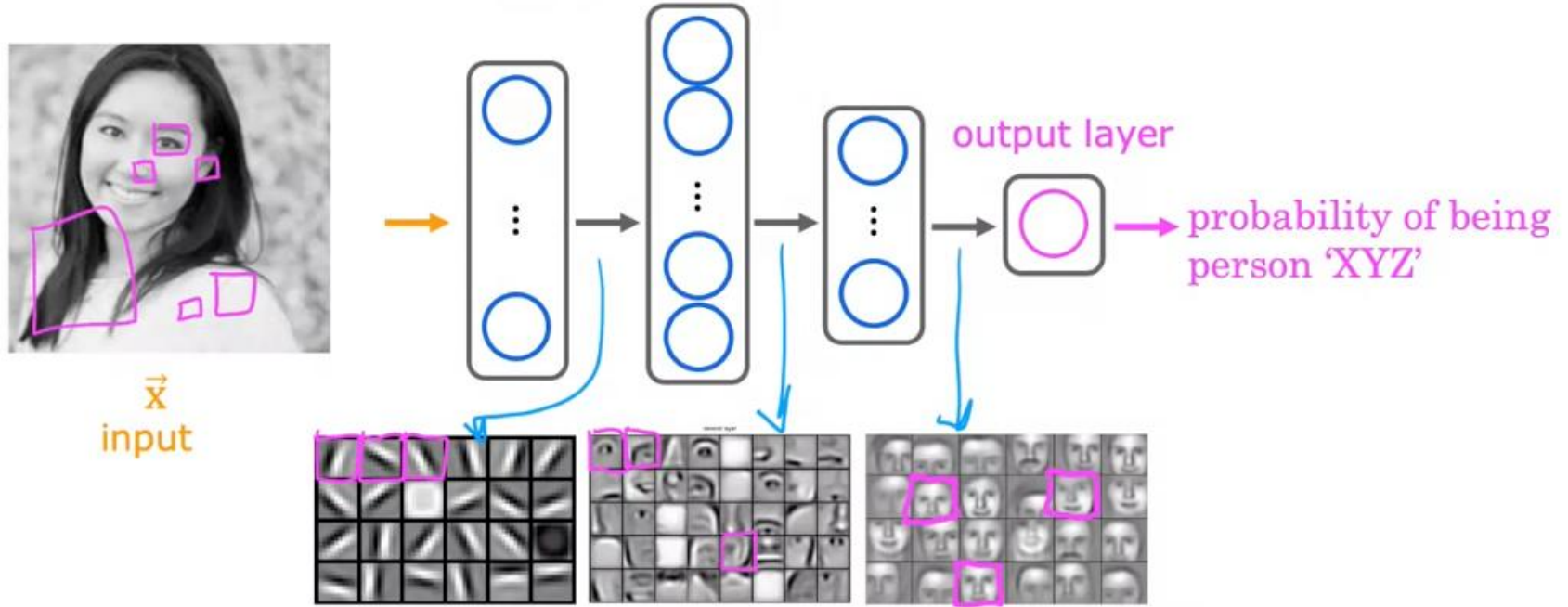
Neural Networks Intuition

Example: Recognizing
images

Face recognition

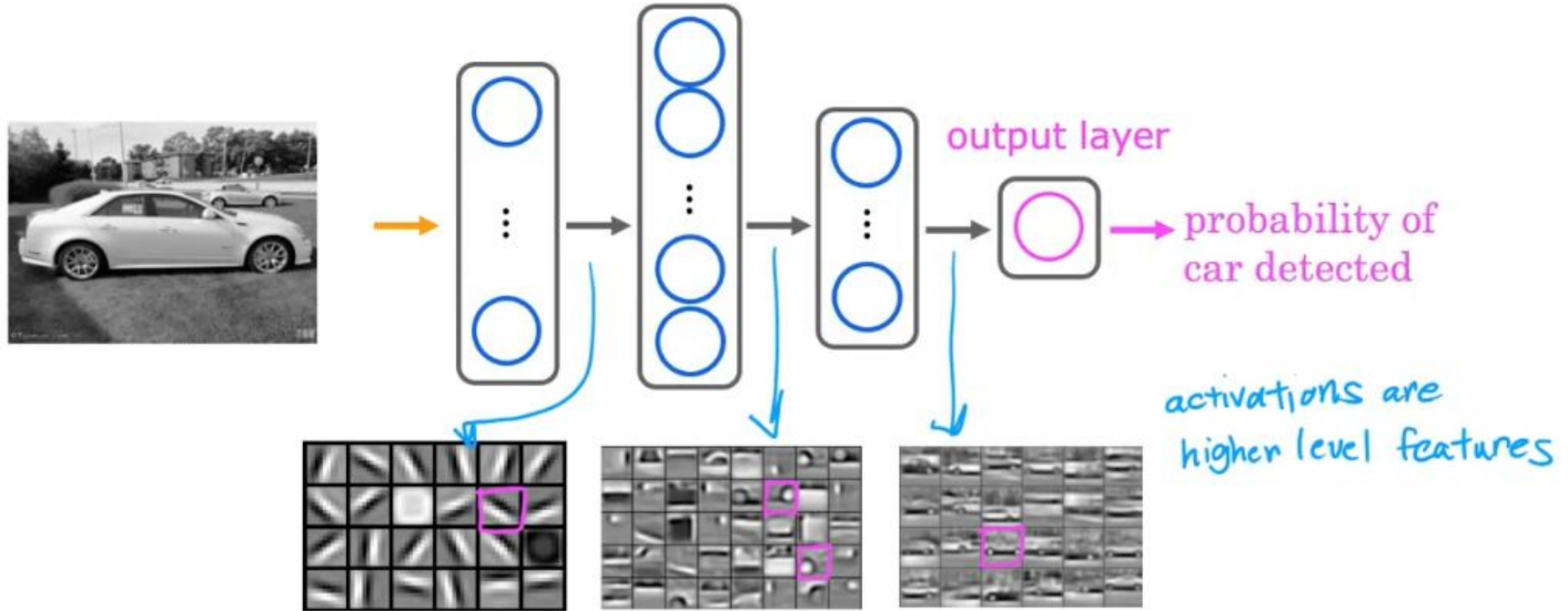


Face recognition

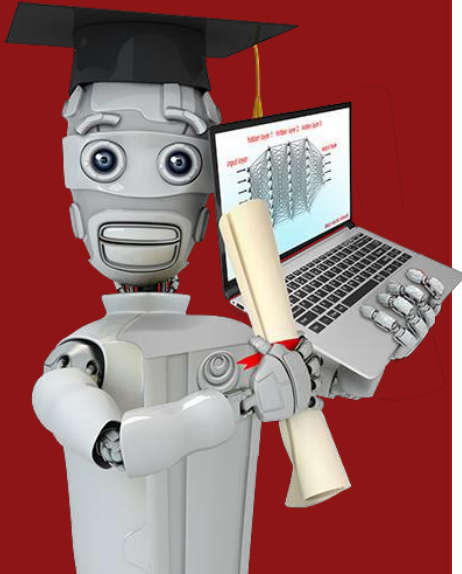


A neural network recognizes faces by turning pixels into numbers and learning features step by step, from simple edges to full face patterns, until it predicts the probability of a specific person.

Car classification



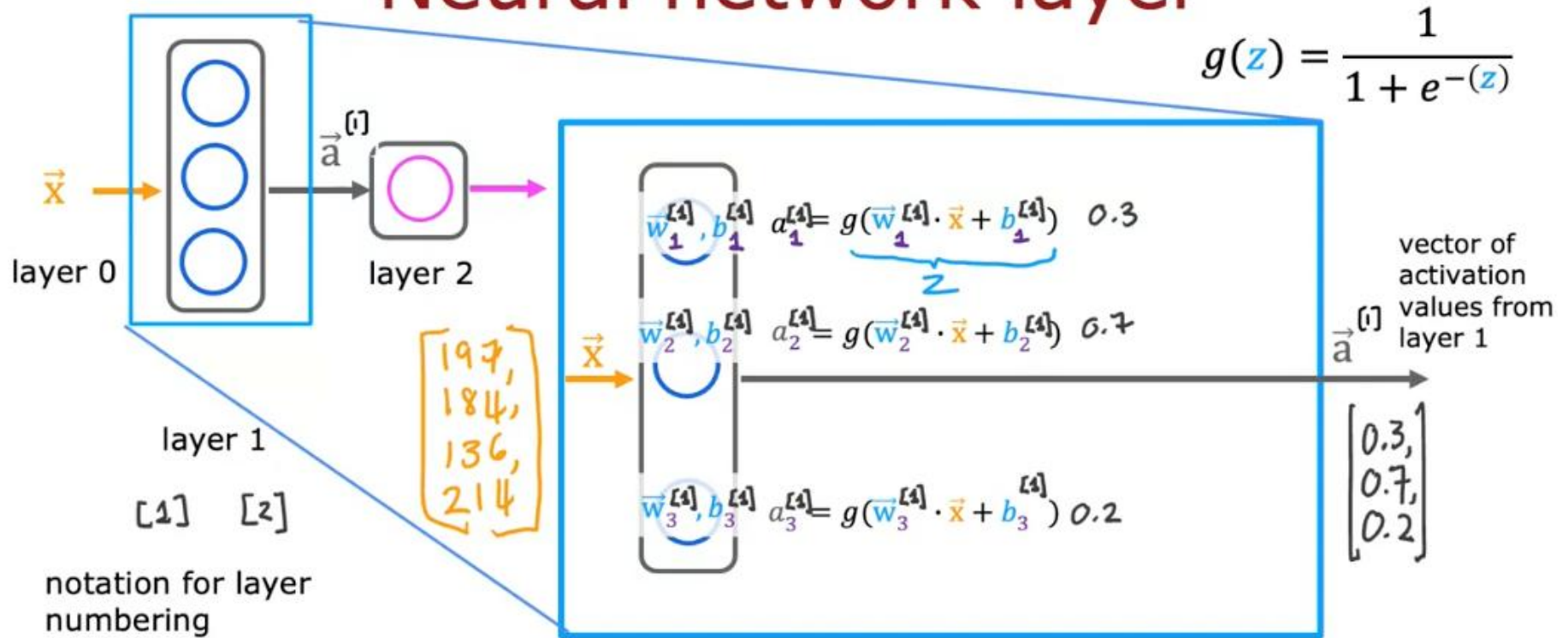
A neural network classifies cars by learning visual features gradually from pixels to car parts, then using these features to predict the probability that a car is present.



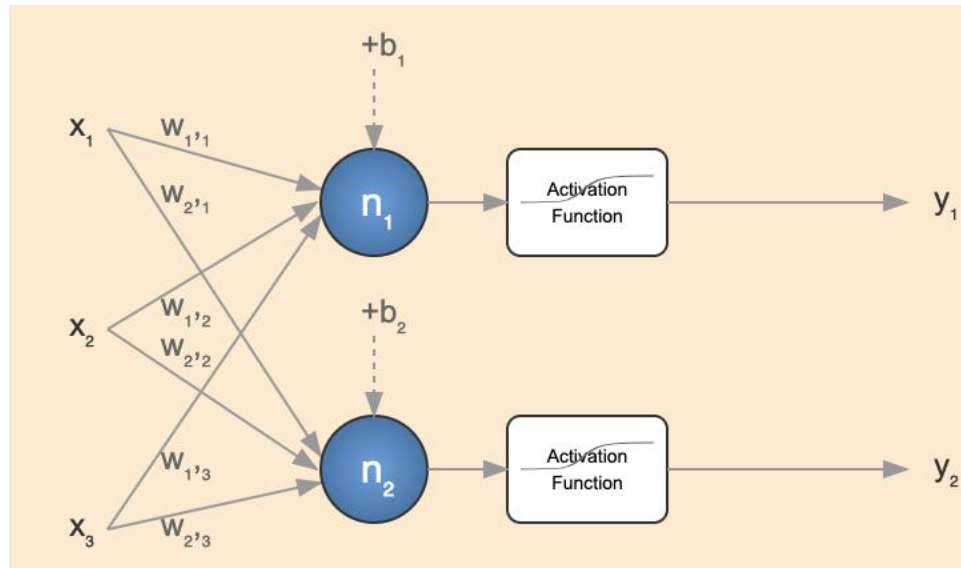
Neural Network Model

Neural Network Layer

Neural network layer

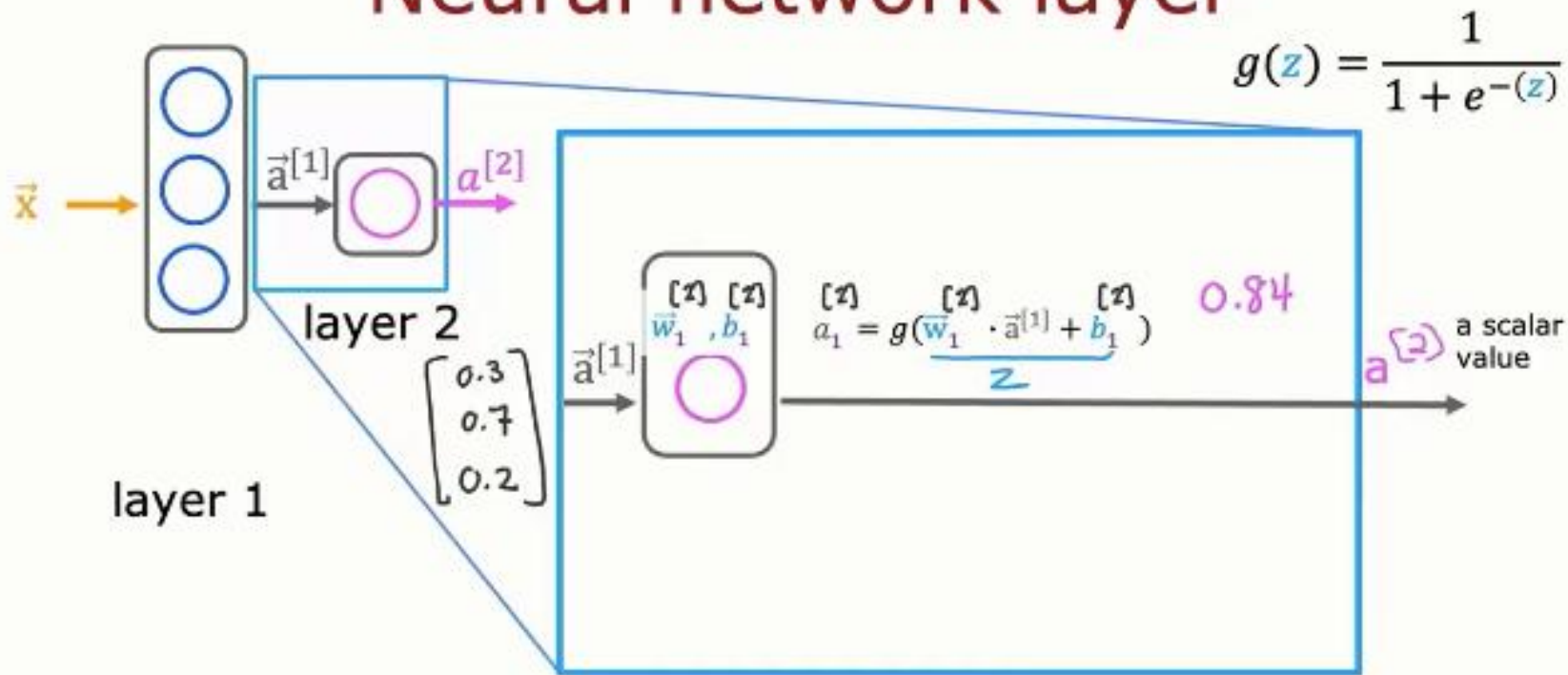


A neural network layer takes an input vector, applies weights, adds bias, uses an activation function, and produces a vector of activation values that is passed to the next layer.



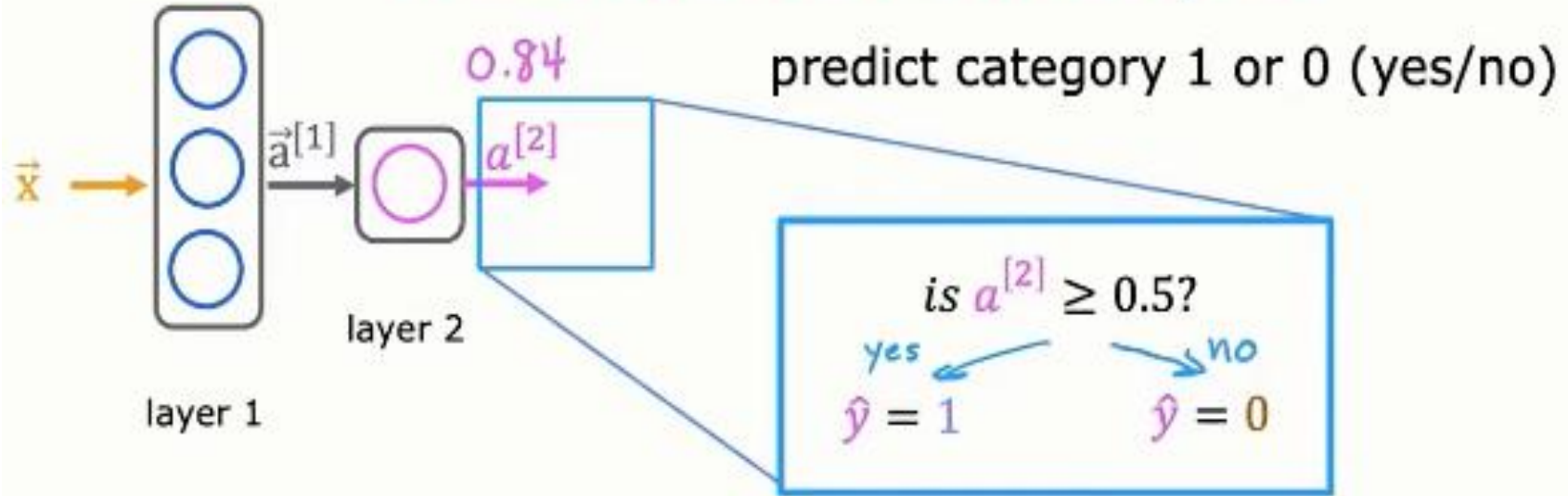
$$\begin{bmatrix} w_{1,1} & w_{1,2} & w_{1,3} \\ w_{2,1} & w_{2,2} & w_{2,3} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} w_{1,1} \cdot x_1 + w_{1,2} \cdot x_2 + w_{1,3} \cdot x_3 \\ w_{2,1} \cdot x_1 + w_{2,2} \cdot x_2 + w_{2,3} \cdot x_3 \end{bmatrix}$$

Neural network layer

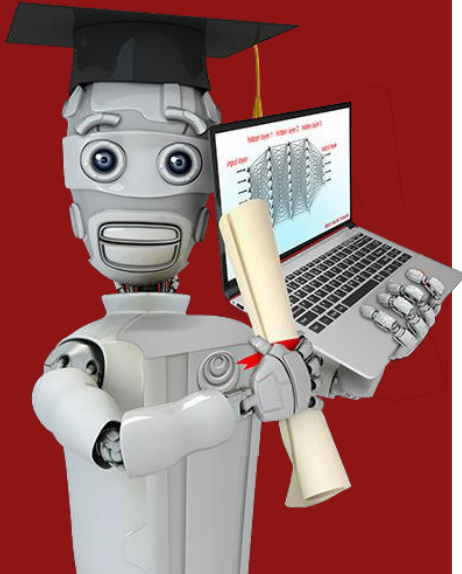


A single neuron in the output layer combines inputs with weights and bias, applies the activation function, and produces one scalar value that represents the final prediction.

Neural network layer



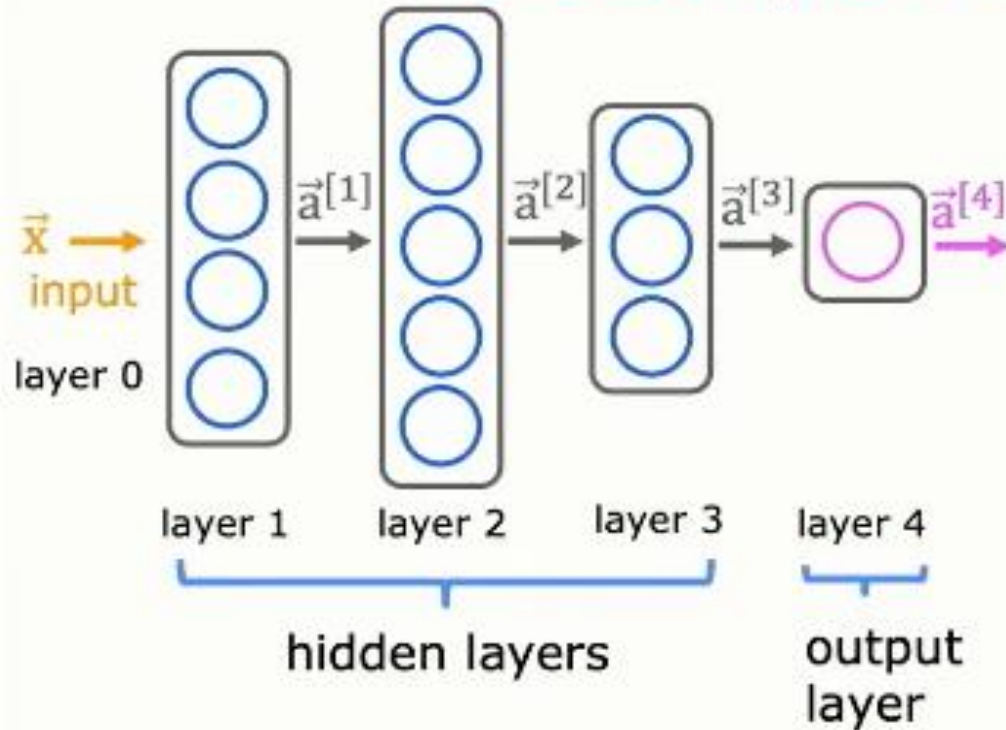
The output layer converts the activation value into a final yes/no decision by comparing the predicted probability to a threshold (such as 0.5).



Neural Network Model

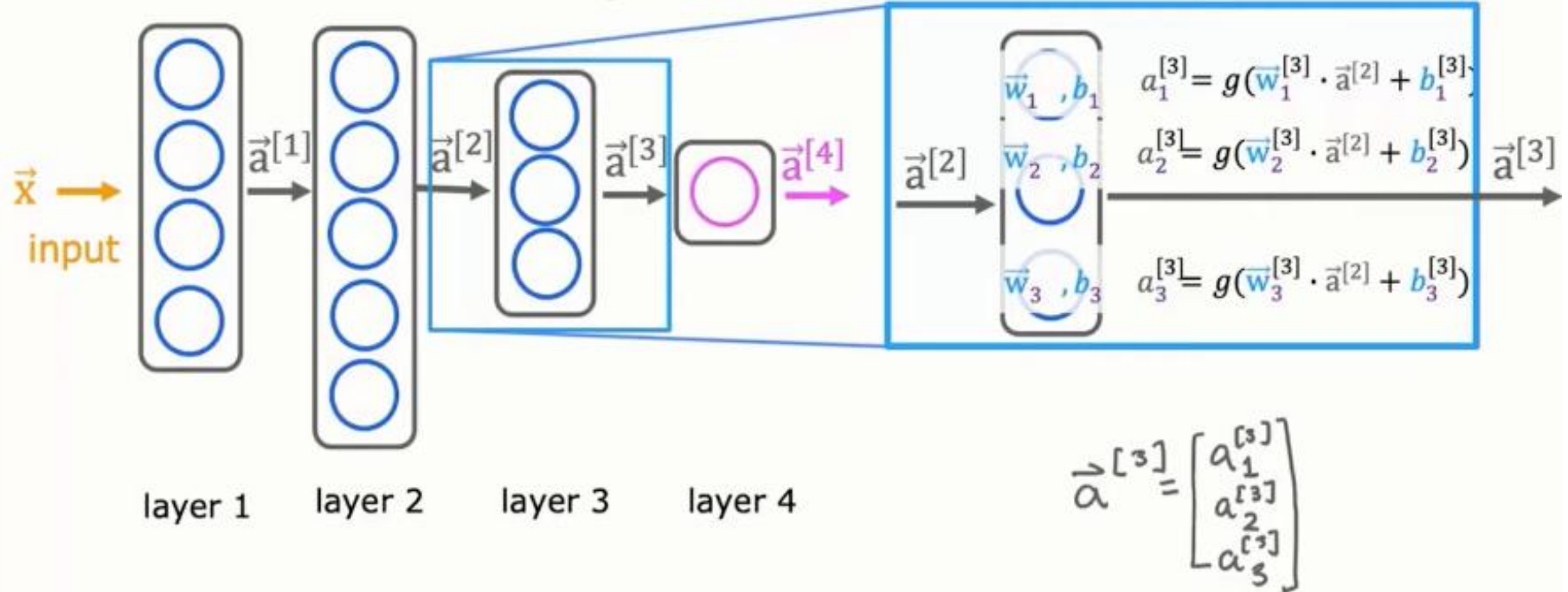
More Complex Neural Networks

More complex neural network



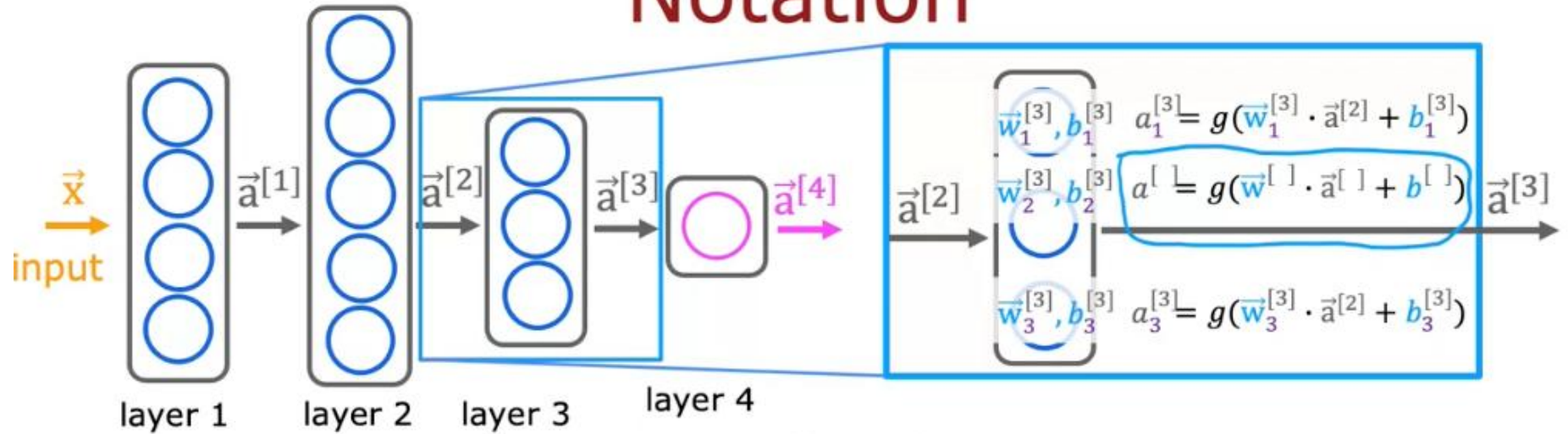
A more complex neural network uses multiple hidden layers to transform the input step by step into higher-level features before producing the final output.

More complex neural network



Each hidden layer applies the same neuron computation in parallel to produce an activation vector, which is passed forward to build deeper and more powerful representations.

Notation

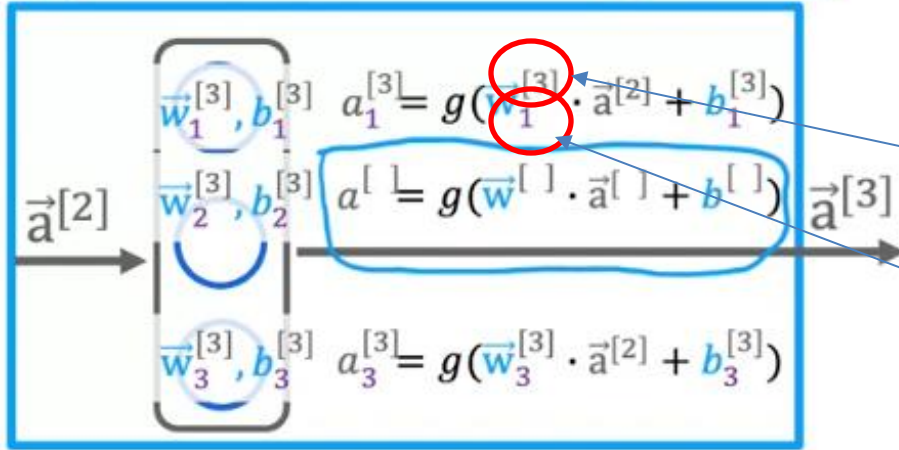


Question:

Can you fill in the superscripts and subscripts for the second neuron?

This slide summarizes neural network notation, showing how activations, weights, and biases are indexed by layer and neuron using subscripts and superscripts.

[In Video Quiz]



Layer number

Nuron number

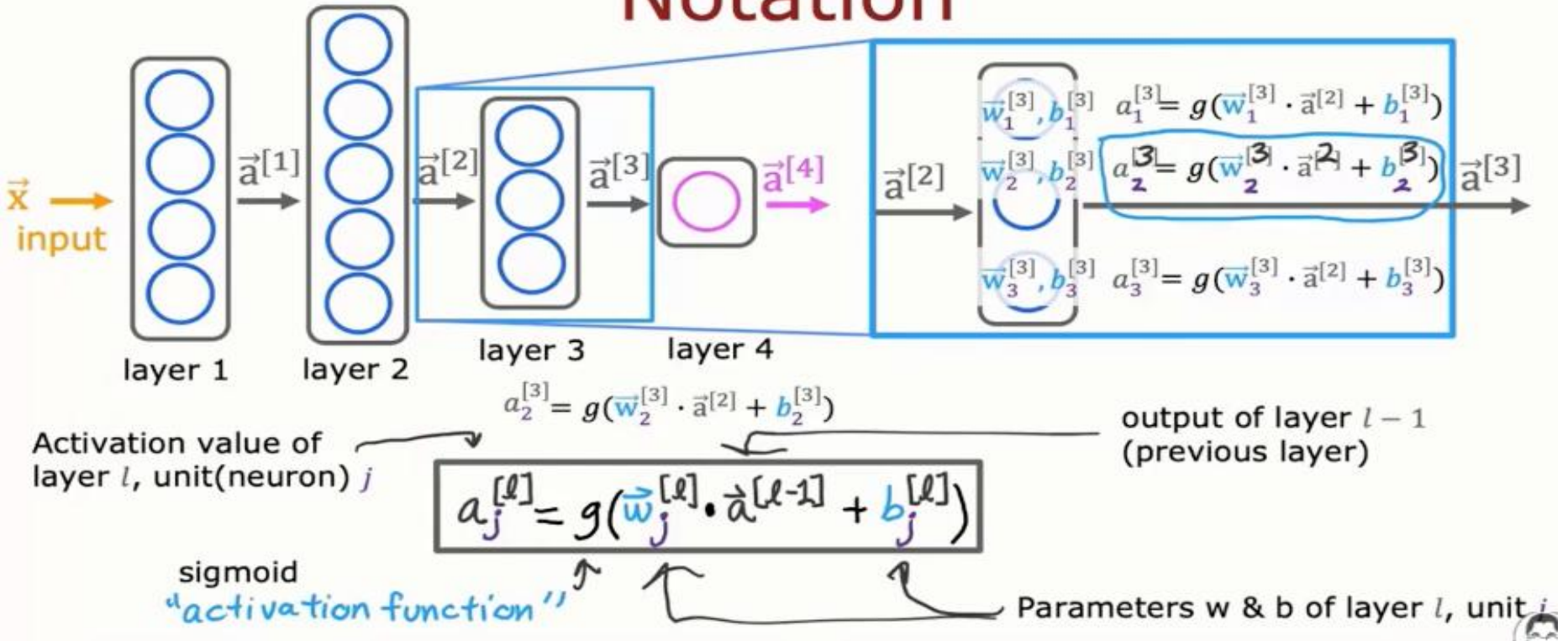
Can you fill in the superscripts and subscripts for the second neuron?

✓ $a_2^{[3]} = g(\vec{w}_2^{[3]} \cdot \vec{a}^{[2]} + b_2^{[3]})$

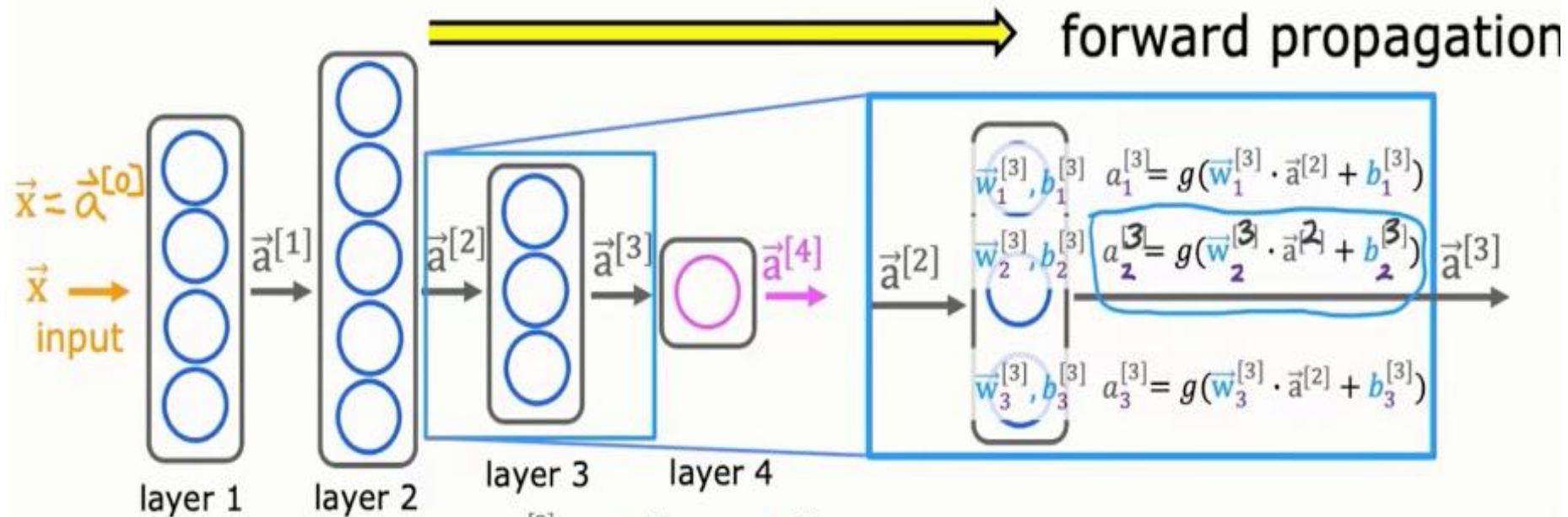
✗ $a_2^{[3]} = g(\vec{w}_2^{[3]} \cdot \vec{a}^{[3]} + b_2^{[3]})$ input is $\vec{a}^{[2]}$

✗ $a_2^{[3]} = g(\vec{w}_2^{[3]} \cdot a_2^{[2]} + b_2^{[3]})$ input is a vector $\vec{a}^{[2]}$ not a single number

Notation



each neuron's activation is computed using weights, bias, and the activation function, with subscripts indicating neuron number and superscripts indicating layer number.



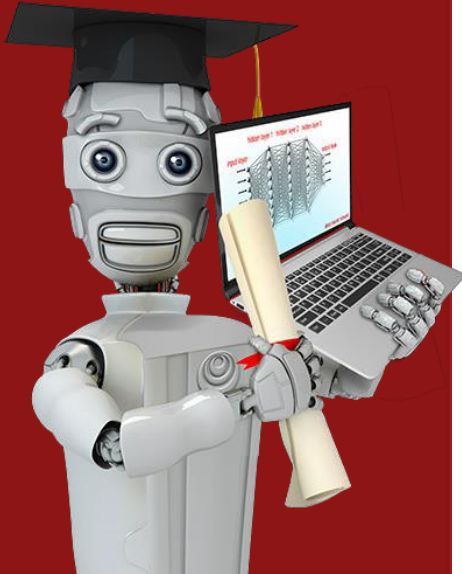
Activation value of layer l , unit(neuron) j

$$a_j^{[l]} = g(\bar{w}_j^{[l]} \cdot \vec{a}^{[l-1]} + b_j^{[l]})$$

output of layer $l-1$ (previous layer)

sigmoid "activation function"

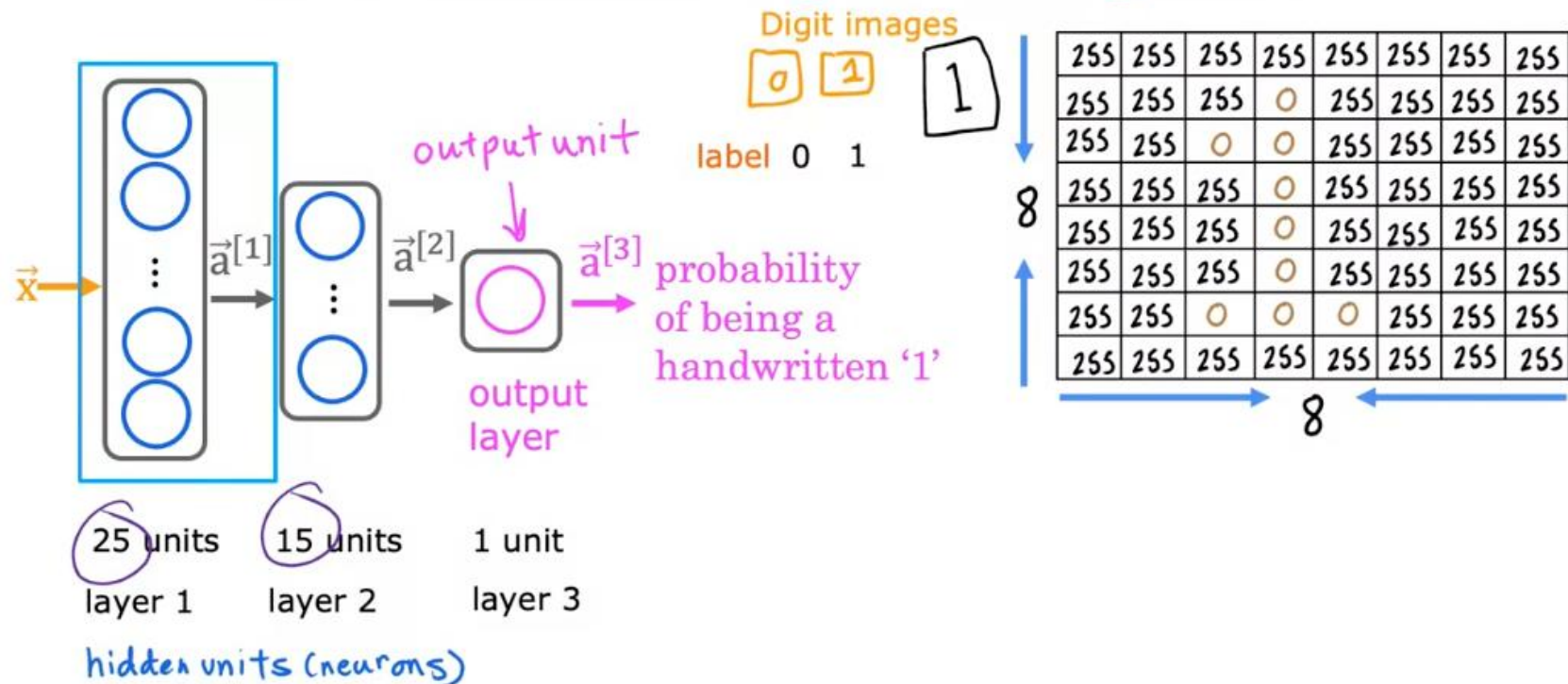
Parameters w & b of layer l , unit j



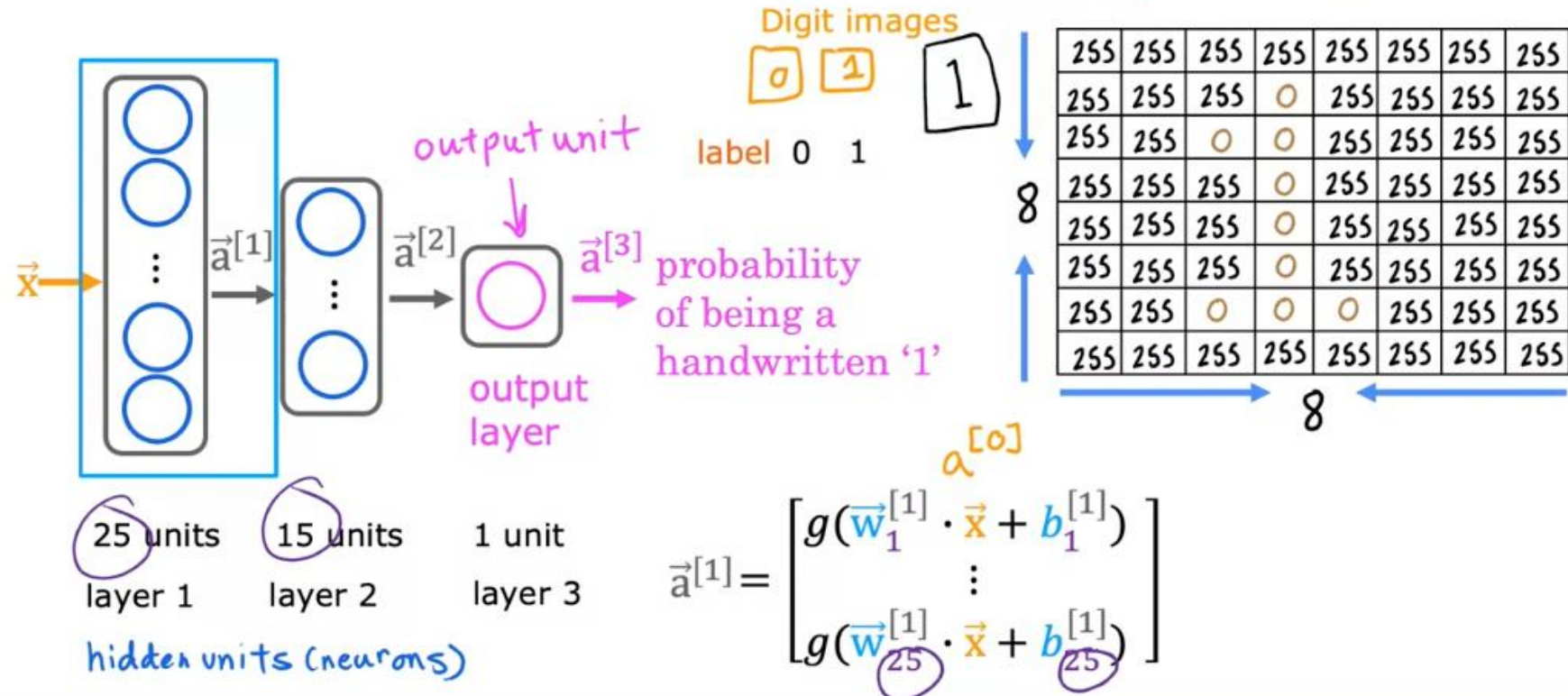
Neural Network Model

Inference: Making Predictions
(forward Propagation)

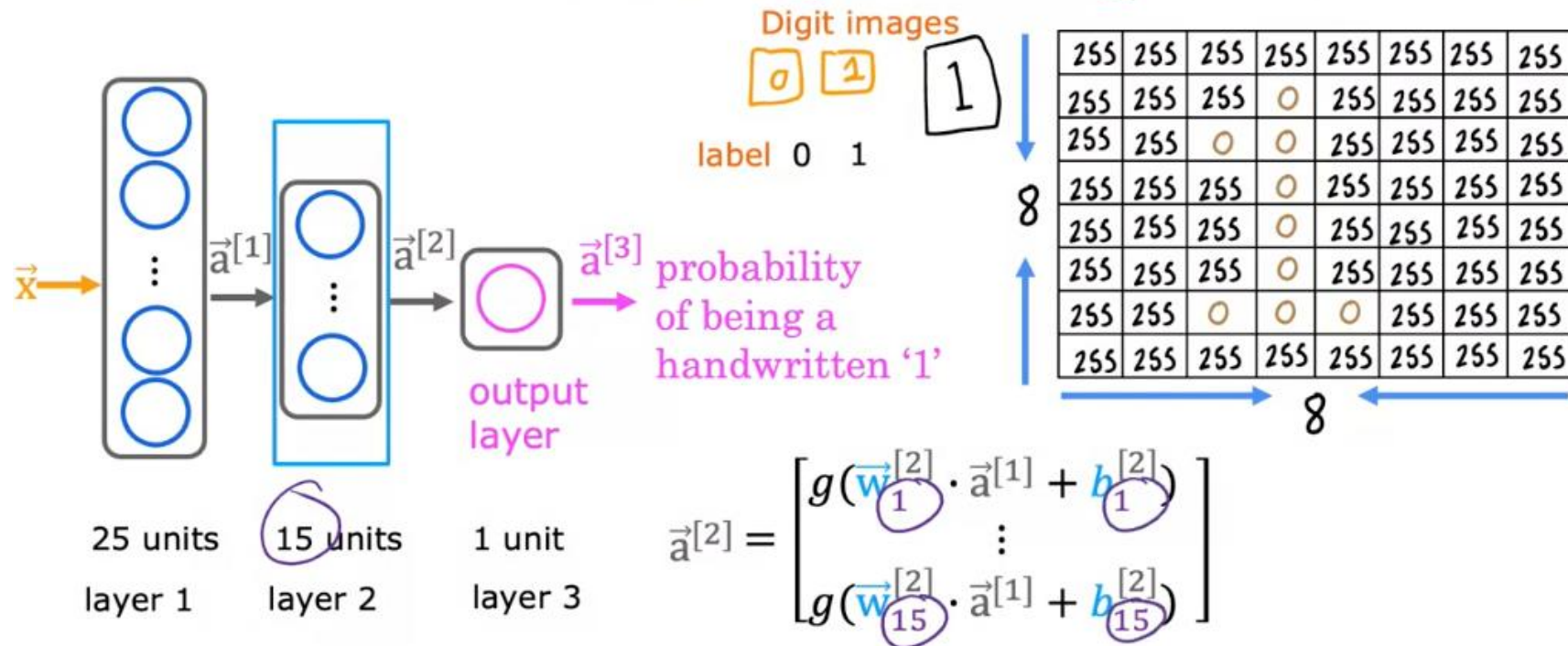
Handwritten digit recognition



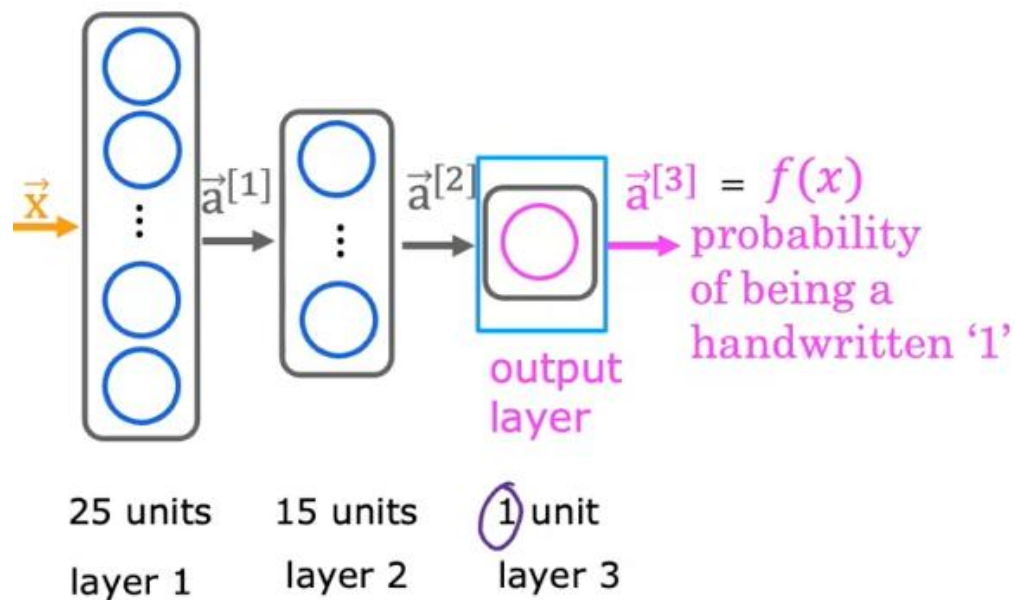
Handwritten digit recognition



Handwritten digit recognition



Handwritten digit recognition



$$\vec{a}^{[3]} = \left[g \left(\vec{w}_1^{[3]} \cdot \vec{a}^{[2]} + b_1^{[3]} \right) \right]$$

is $a_1^{[3]} \geq 0.5$?

yes

no

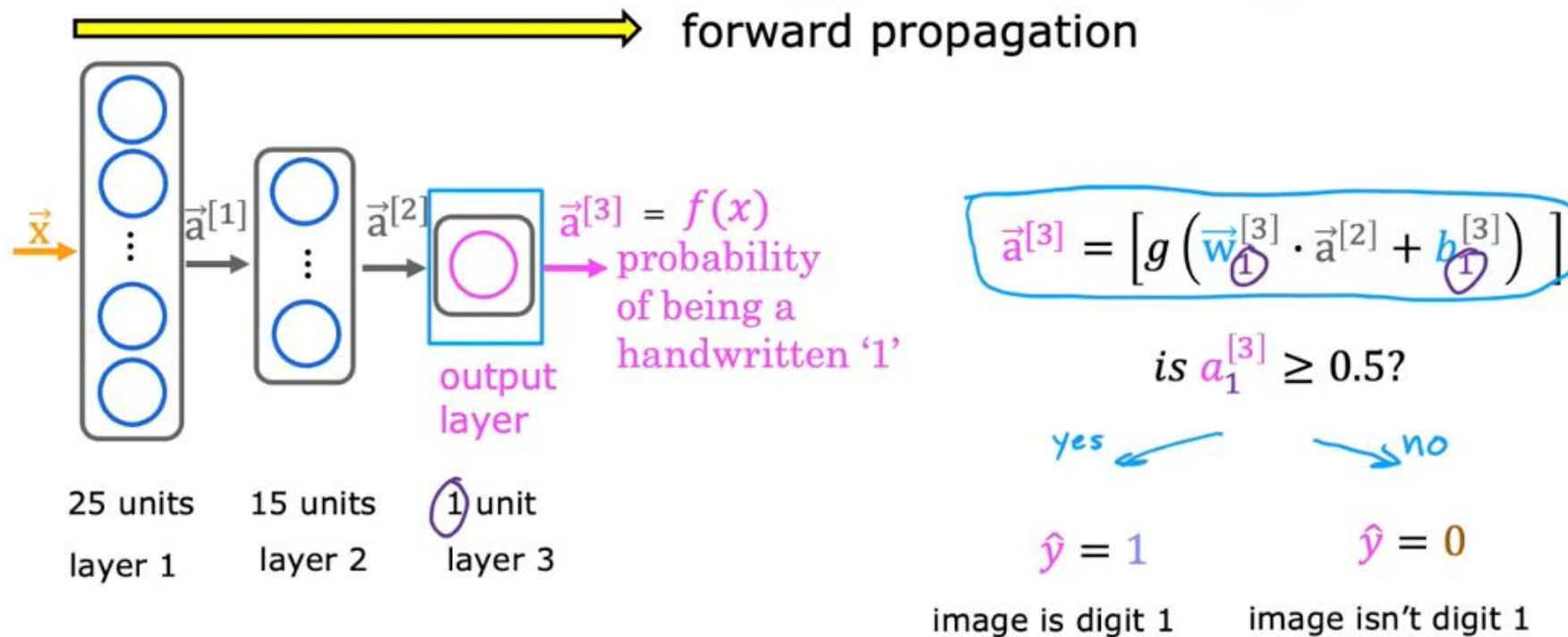
$$\hat{y} = 1$$

$$\hat{y} = 0$$

image is digit 1

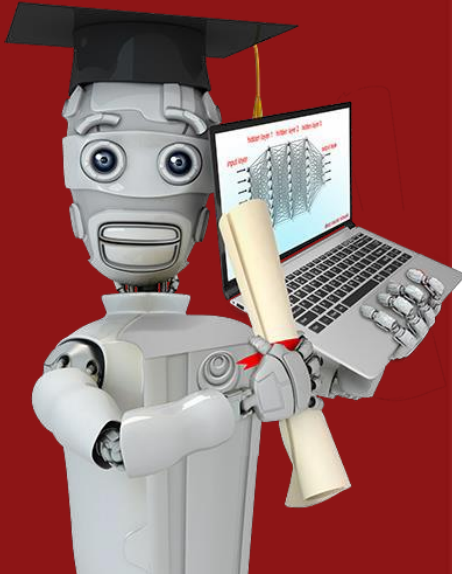
image isn't digit 1

Handwritten digit recognition





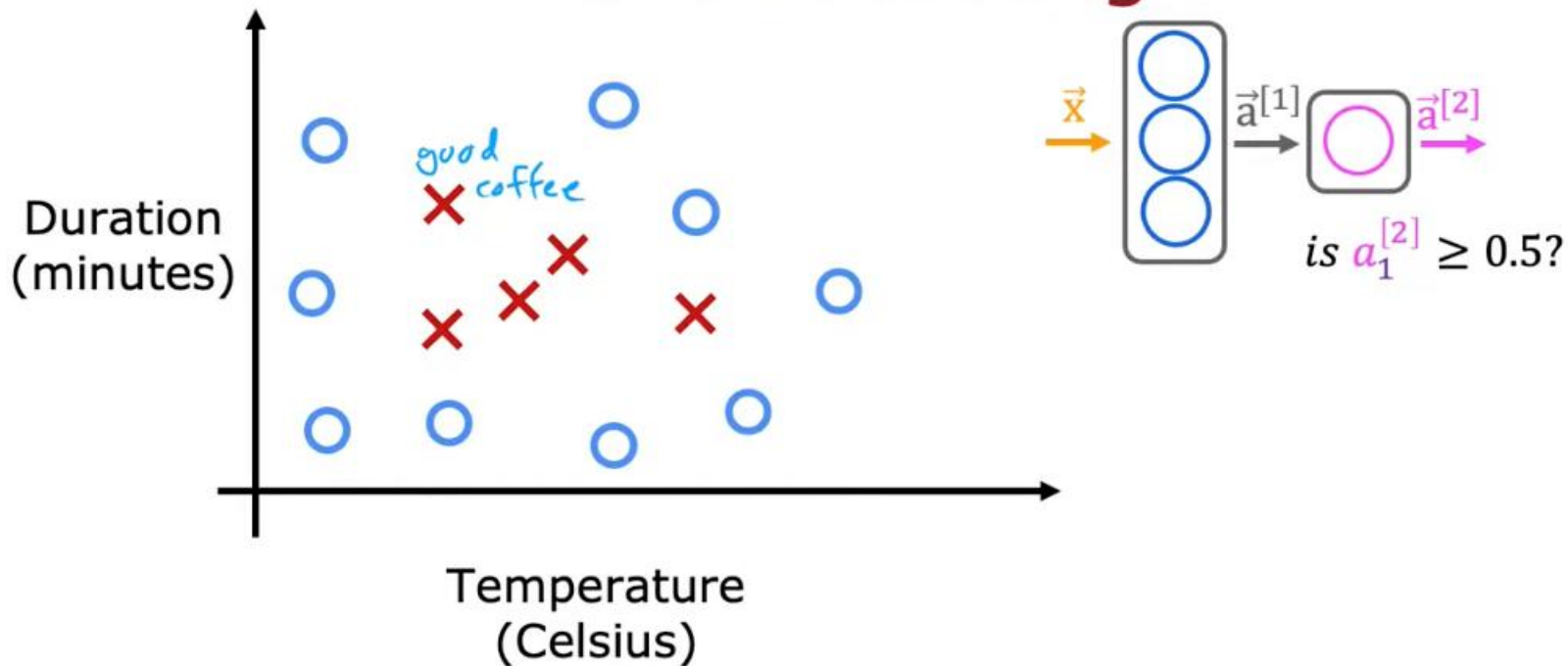
Stanford
ONLINE



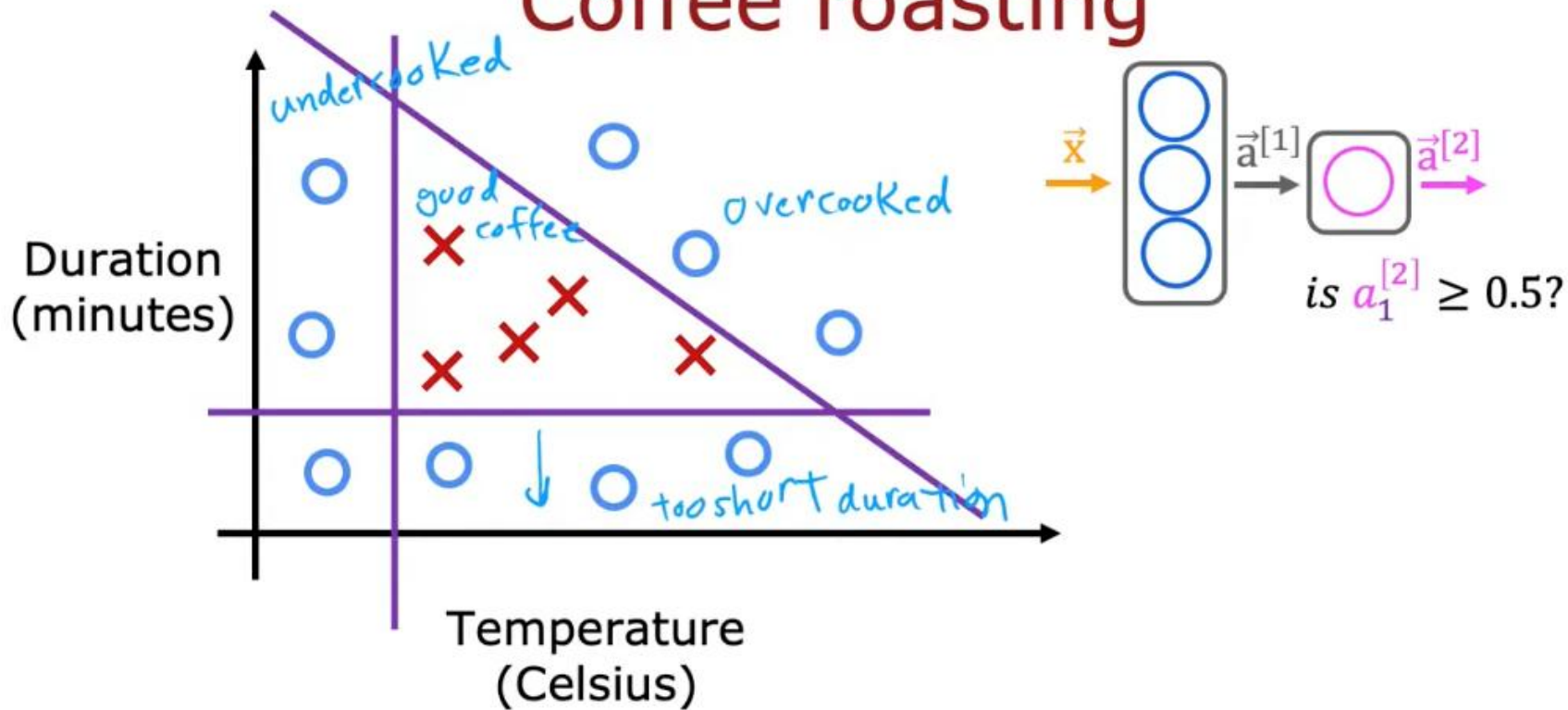
TensorFlow Implementation

Inference in Code

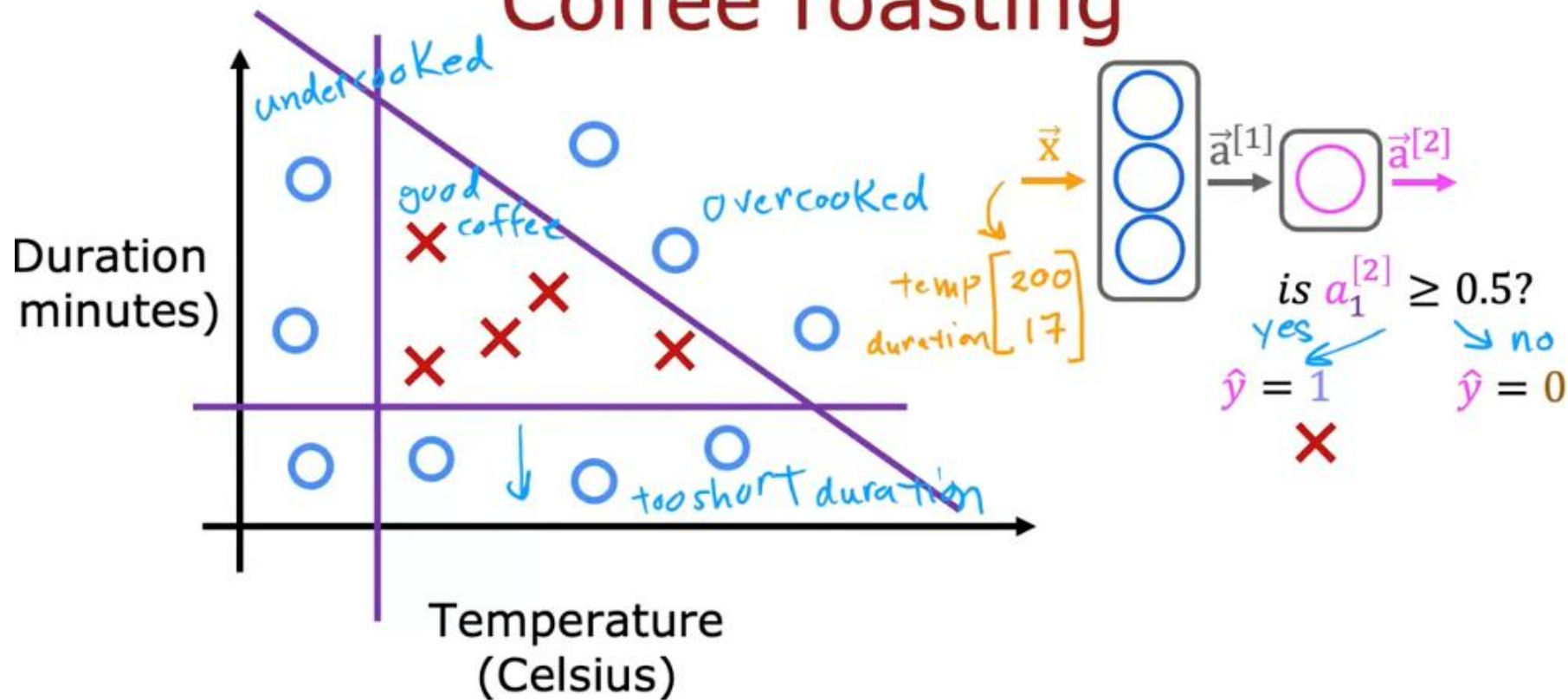
Coffee roasting



Coffee roasting

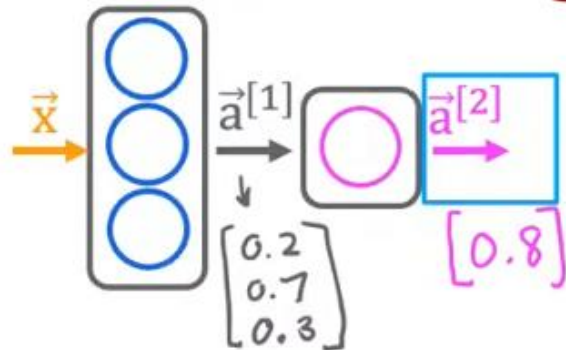


Coffee roasting



Build the model using TensorFlow

Dense is a **layer class**
When you call it, you
**create an object (an
instance)** of that class



is $a_1^{[2]} \geq 0.5$?
yes $\hat{y} = 1$ $\times$
no $\hat{y} = 0$ $\circ$

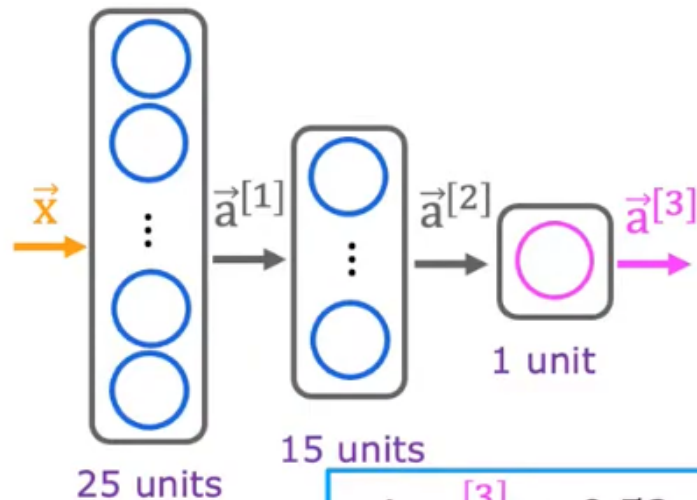
```
if a2 >= 0.5:  
    yhat = 1  
else:  
    yhat = 0
```

```
x = np.array([[200.0, 17.0]])  
layer_1 = Dense(units=3, activation='sigmoid')  
a1 = layer_1(x)
```

```
layer_2 = Dense(units=1, activation='sigmoid')  
a2 = layer_2(a1)
```

Dense layer object

Model for digit classification



```
x = np.array([[0.0,...245,...240...0]])  
layer_1 = Dense(units=25, activation='sigmoid')  
a1 = layer_1(x)
```

```
layer_2 = Dense(units=15, activation='sigmoid')  
a2 = layer_2(a1)
```

```
layer_3 = Dense(units=1, activation='sigmoid')  
a3 = layer_3(a2)
```

is $a_1^{[3]} \geq 0.5$?

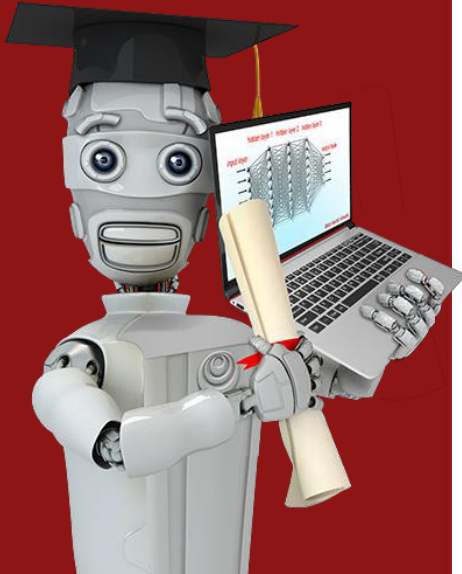
$\hat{y} = 1$

✗

$\hat{y} = 0$

○

```
if a3 >= 0.5:  
    yhat = 1  
else:  
    yhat = 0
```



TensorFlow Implementation

Data in TensorFlow

Feature vectors

temperature (Celsius)	duration (minutes)	Good coffee? (1/0)
200.0	17.0	1
425.0	18.5	0
...	...	...

`x = np.array([[200.0, 17.0]])` ←
`[[200.0, 17.0]]`

why?

Note about numpy arrays

3 columns
2 rows
 $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$
2 x 3 matrix

4 rows
2 columns
 $\begin{bmatrix} 0.1 & 0.2 \\ -3 & -4 \\ -0.5 & -0.6 \\ 7 & 8 \end{bmatrix}$
4 x 2 matrix

```
x = np.array([[1, 2, 3],  
              [4, 5, 6]])
```

1st row
2nd row

2D array

$\begin{bmatrix} [1, 2, 3], \\ [4, 5, 6] \end{bmatrix}$

```
x = np.array([[0.1, 0.2],  
              [-3.0, -4.0],  
              [-0.5, -0.6],  
              [7.0, 8.0]])
```

$\begin{bmatrix} [0.1, 0.2], \\ [-3.0, -4.0], \\ [-0.5, -0.6], \\ [7.0, 8.0] \end{bmatrix}$

2 x 3

4 x 2

1 x 2

2 x 1

Note about numpy arrays

`x = np.array([[200, 17]])` → $\begin{bmatrix} 200 & 17 \end{bmatrix}$ 1 x 2 Row Vector

TensorFlow

`x = np.array([[200], [17]])` $\begin{bmatrix} 200 \\ 17 \end{bmatrix}$ 2 x 1 Column Vector

Linear
Regression

→ `x = np.array([200, 17])`

1D
"Vector"

Linear Array with no rows or columns, and
Can not be used as input for NN

Feature vectors

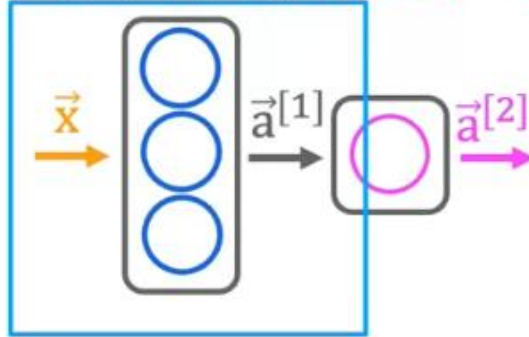
temperature (Celsius)	duration (minutes)	Good coffee? (1/0)
200.0	17.0	1
425.0	18.5	0
...	...	...

`x = np.array([[200.0, 17.0]])` ←

`[[200.0, 17.0]]`

↓ ↓ 1×2
→ `[200.0 17.0]`

Activation vector



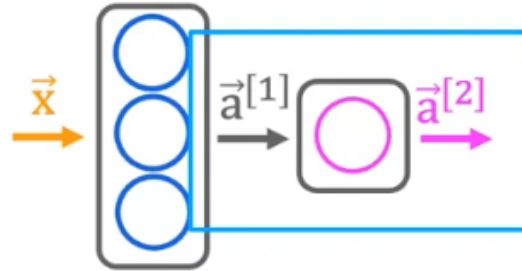
```
x = np.array([[200.0, 17.0]])  
layer_1 = Dense(units=3, activation='sigmoid')  
a1 = layer_1(x)
```

Tensor is a data type in TensorFlow, created for Numerical data

→ $\begin{bmatrix} 0.2 & 0.7 & 0.3 \end{bmatrix}$ 1 x 3 matrix
`tf.Tensor([[0.2 0.7 0.3]], shape=(1, 3), dtype=float32)`

→ `a1.numpy()` In TensorFlow, **all data (inputs, activations, weights, outputs)** are stored as **tensors**
`array([[0.2, 0.7, 0.3]], dtype=float32)`

Activation vector



```
> layer_2 = Dense(units=1, activation='sigmoid')  
> a2 = layer_2(a1)
```

↪ $[[0.8]]$ ←

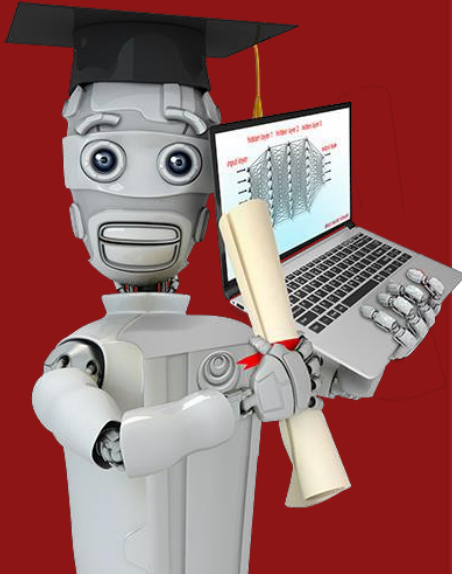
1 x 1

→ `tf.Tensor([[0.8]], shape=(1, 1), dtype=float32)`

```
> a2.numpy()
```

↪ Convert tensor to numpy matrix

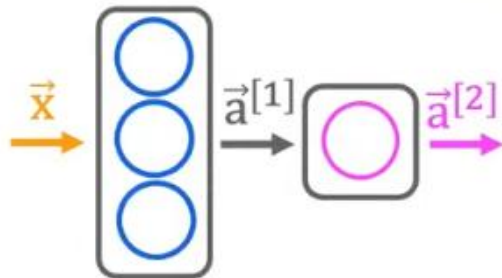
→ `array([[0.8]], dtype=float32)`



TensorFlow Implementation

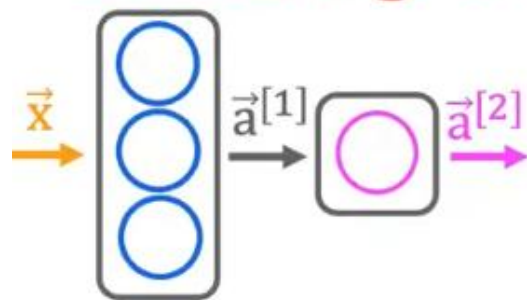
Building a Neural Network

What you saw earlier



```
→ x = np.array([[200.0, 17.0]])  
→ layer_1 = Dense(units=3, activation="sigmoid")  
→ a1 = layer_1(x)  
  
→ layer_2 = Dense(units=1, activation="sigmoid")  
→ a2 = layer_2(a1)
```

Building a neural network architecture



```
→ layer_1 = Dense(units=3, activation="sigmoid") ←
→ layer_2 = Dense(units=1, activation="sigmoid") ←
→ model = Sequential([layer_1, layer_2])
```

```
x = np.array([[200.0, 17.0],
               [120.0, 5.0],
               [425.0, 20.0],
               [212.0, 18.0]])
```

4 x 2

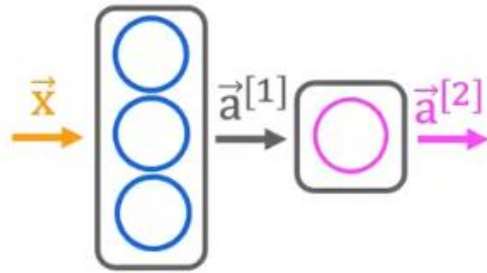
		y
200	17	1
120	5	0
425	20	0
212	18	1

targets `y = np.array([1,0,0,1])`

```
model.compile(...) ← more about this next week!
model.fit(x,y)
```

```
→ model.predict(x_new) ←
```

Building a neural network architecture



Alternative Way of defining layers

```
→ model = Sequential([  
    → Dense(units=3, activation="sigmoid"),  
    → Dense(units=1, activation="sigmoid")])
```

		y
200	17	1
120	5	0
425	20	0
212	18	1

```
x = np.array([[200.0, 17.0],  
              [120.0, 5.0],  
              [425.0, 20.0],  
              [212.0, 18.0]])
```

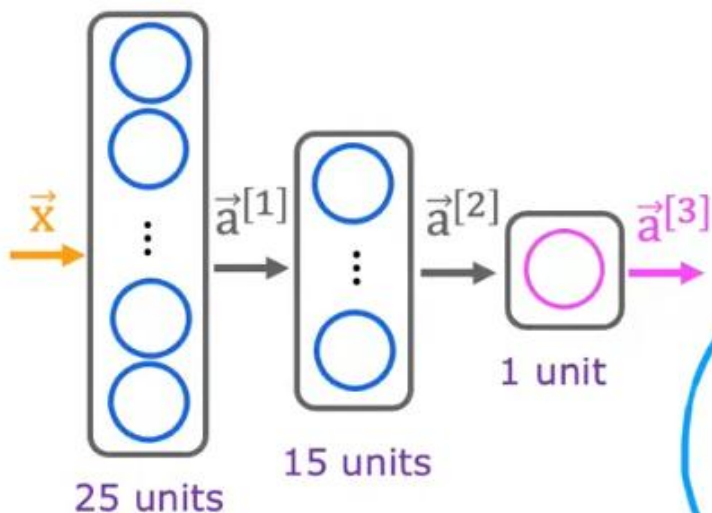
4 x 2

targets `y = np.array([1,0,0,1])`

```
model.compile(...) ← more about this next week!  
model.fit(x,y)
```

```
→ model.predict(x_new) ←
```

Digit classification model

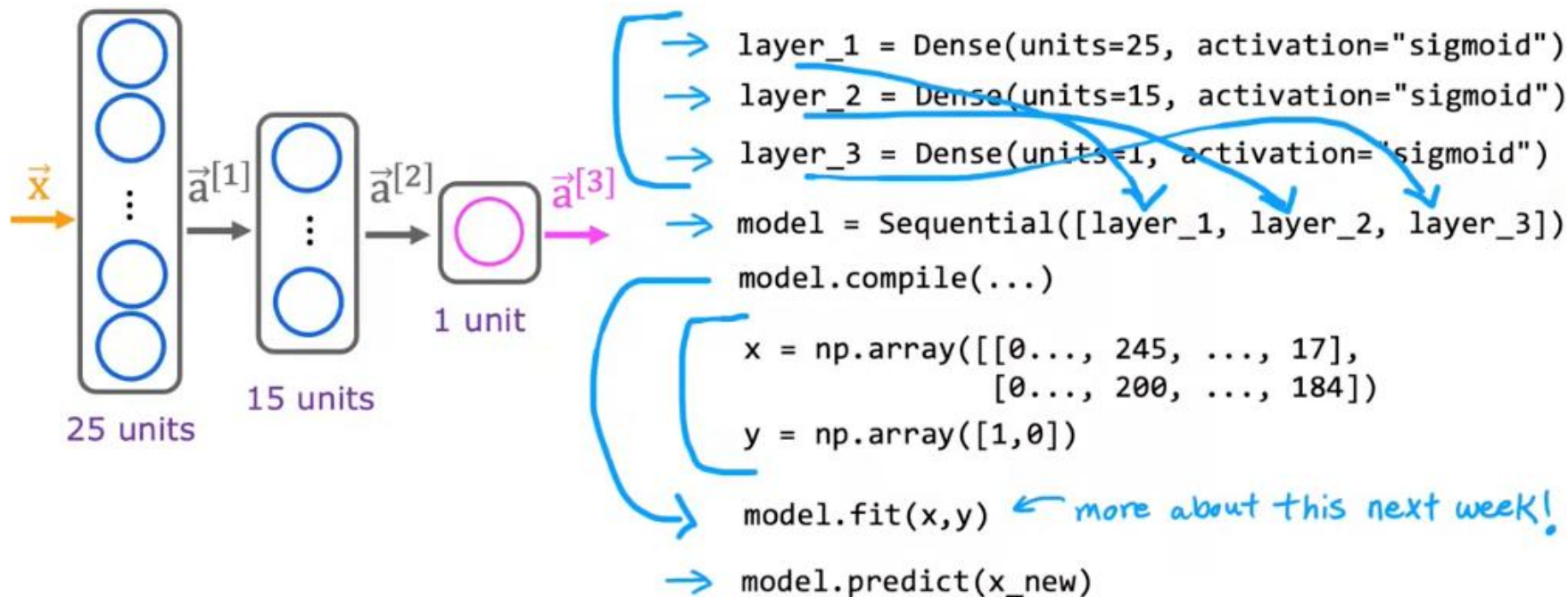


```
→ layer_1 = Dense(units=25, activation="sigmoid")
→ layer_2 = Dense(units=15, activation="sigmoid")
→ layer_3 = Dense(units=1, activation="sigmoid")
→ model = Sequential([layer_1, layer_2, layer_3])
→ model.compile(...)

x = np.array([[0..., 245, ..., 17],
              [0..., 200, ..., 184]])
y = np.array([1,0])

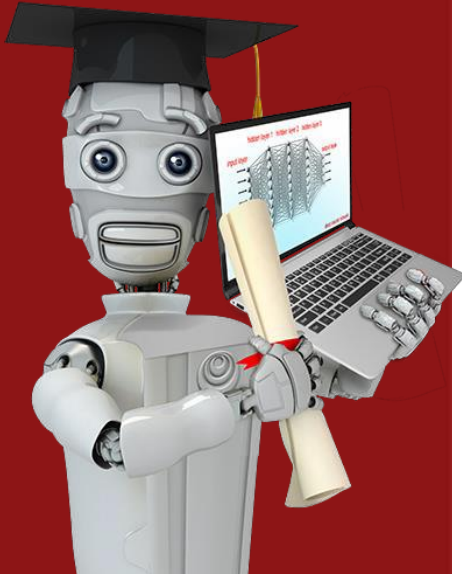
→ model.fit(x,y) ← more about this next week!
→ model.predict(x_new)
```

Digit classification model





Stanford
ONLINE

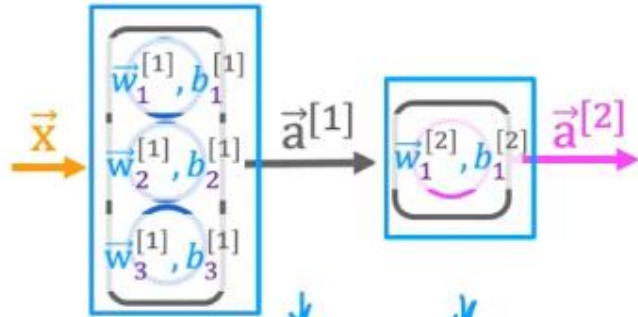


Neural Network Implementation in Python

Forward Prop in a single Layer

Andrew Ng

forward prop (coffee roasting model)



`x = np.array([200, 17])`

1D arrays

$$a_1^{[1]} = g(\vec{w}_1^{[1]} \cdot \vec{x} + b_1^{[1]})$$

```
w1_1 = np.array([1, 2])
```

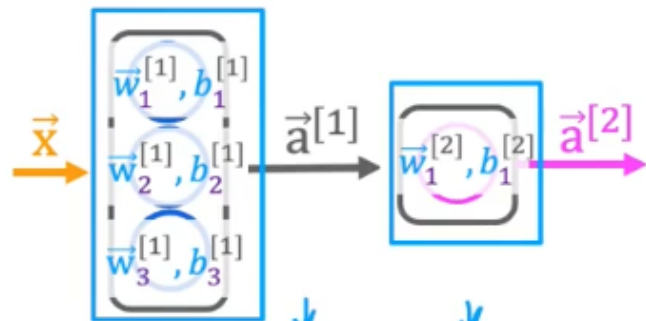
```
b1_1 = np.array([-1])
```

```
z1_1 = np.dot(w1_1, x) + b1_1
```

```
a1_1 = sigmoid(z1_1)
```

$w_1^{[2]}$ `w2_1`

forward prop (coffee roasting model)



$x = \text{np.array}([200, 17])$

$$a_1^{[1]} = g(\bar{w}_1^{[1]} \cdot \vec{x} + b_1^{[1]})$$

$w1_1 = \text{np.array}([1, 2])$

$b1_1 = \text{np.array}([-1])$

$z1_1 = \text{np.dot}(w1_1, x) + b1_1$

$a1_1 = \text{sigmoid}(z1_1)$

1D arrays

$$a_2^{[1]} = g(\bar{w}_2^{[1]} \cdot \vec{x} + b_2^{[1]})$$

$w1_2 = \text{np.array}([-3, 4])$

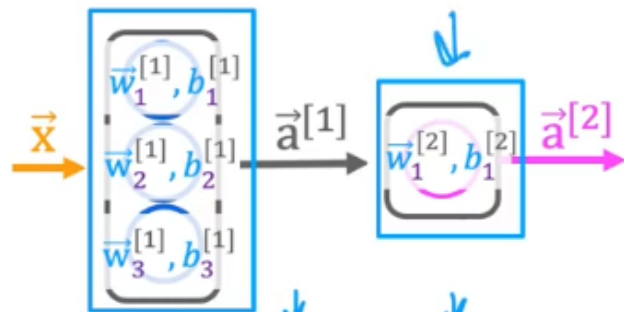
$b1_2 = \text{np.array}([1])$

$z1_2 = \text{np.dot}(w1_2, x) + b1_2$

$a1_2 = \text{sigmoid}(z1_2)$

$w_1^{[2]}$ $w2_1$

forward prop (coffee roasting model)



$x = \text{np.array}([200, 17])$

1D arrays

$$a_1^{[1]} = g(\bar{w}_1^{[1]} \cdot \vec{x} + b_1^{[1]})$$

$$a_2^{[1]} = g(\bar{w}_2^{[1]} \cdot \vec{x} + b_2^{[1]})$$

$w1_1 = \text{np.array}([1, 2])$

$w1_2 = \text{np.array}([-3, 4])$

$b1_1 = \text{np.array}([-1])$

$b1_2 = \text{np.array}([1])$

$z1_1 = \text{np.dot}(w1_1, x) + b1_1$

$z1_2 = \text{np.dot}(w1_2, x) + b1_2$

$a1_1 = \text{sigmoid}(z1_1)$

$a1_2 = \text{sigmoid}(z1_2)$

$$a_3^{[1]} = g(\bar{w}_3^{[1]} \cdot \vec{x} + b_3^{[1]})$$

$w1_3 = \text{np.array}([5, -6])$

$b1_3 = \text{np.array}([2])$

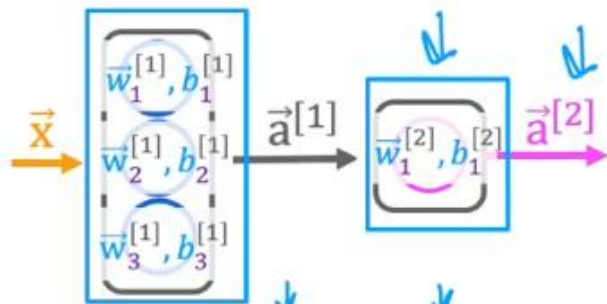
$z1_3 = \text{np.dot}(w1_3, x) + b1_3$

$a1_3 = \text{sigmoid}(z1_3)$

$a1 = \text{np.array}([a1_1, a1_2, a1_3])$

$w_1^{[2]}$ $w2_1$

forward prop (coffee roasting model)



$x = \text{np.array}([200, 17])$

1D arrays

$$a_1^{[1]} = g(\bar{w}_1^{[1]} \cdot \bar{x} + b_1^{[1]})$$

$$a_2^{[1]} = g(\bar{w}_2^{[1]} \cdot \bar{x} + b_2^{[1]})$$

$$a_3^{[1]} = g(\bar{w}_3^{[1]} \cdot \bar{x} + b_3^{[1]})$$

$w1_1 = \text{np.array}([1, 2])$

$w1_2 = \text{np.array}([-3, 4])$

$w1_3 = \text{np.array}([5, -6])$

$b1_1 = \text{np.array}([-1])$

$b1_2 = \text{np.array}([1])$

$b1_3 = \text{np.array}([2])$

$z1_1 = \text{np.dot}(w1_1, x) + b1_1$

$z1_2 = \text{np.dot}(w1_2, x) + b1_2$

$z1_3 = \text{np.dot}(w1_3, x) + b1_3$

$a1_1 = \text{sigmoid}(z1_1)$

$a1_2 = \text{sigmoid}(z1_2)$

$a1_3 = \text{sigmoid}(z1_3)$

$a1 = \text{np.array}([a1_1, a1_2, a1_3])$

$$a_1^{[2]} = g(\bar{w}_1^{[2]} \cdot \bar{a}^{[1]} + b_1^{[2]})$$

$w2_1 = \text{np.array}([-7, 8, 9])$

$b2_1 = \text{np.array}([3])$

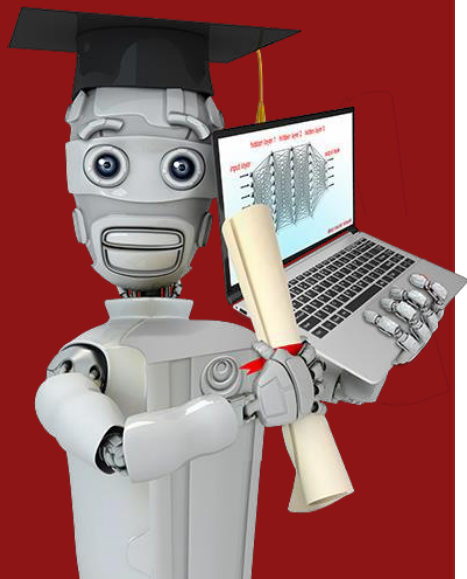
$z2_1 = \text{np.dot}(w2_1, a1) + b2_1$

$a2_1 = \text{sigmoid}(z2_1)$

$w_1^{[2]}$ $w2_1$



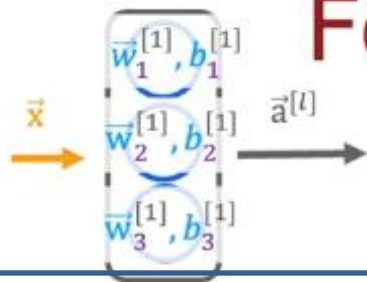
Stanford
ONLINE



Neural network implementation in Python

General implementation of
forward propagation

Forward prop in NumPy



$$\vec{w}_1^{[1]} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad \vec{w}_2^{[1]} = \begin{bmatrix} -3 \\ 4 \end{bmatrix} \quad \vec{w}_3^{[1]} = \begin{bmatrix} 5 \\ -6 \end{bmatrix}$$

```
W = np.array([  
    [1, -3, 5]  
    [2, 4, -6]]) 2 by 3
```

$$b_1^{[l]} = -1 \quad b_2^{[l]} = 1 \quad b_3^{[l]} = 2$$

```
b = np.array([-1, 1, 2])
```

$$\vec{a}^{[0]} = \vec{x}$$

```
a_in = np.array([-2, 4])
```

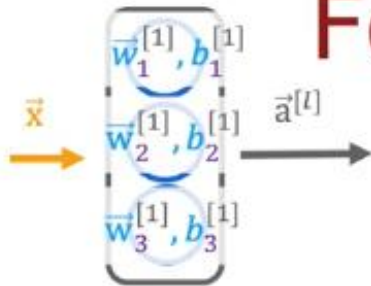
def dense(a_in, W, b): How to define the function dense ()
That we used before

Define W

Define b

Define the activation a_in

Forward prop in NumPy



$$\bar{w}_1^{[1]} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad \bar{w}_2^{[1]} = \begin{bmatrix} -3 \\ 4 \end{bmatrix} \quad \bar{w}_3^{[1]} = \begin{bmatrix} 5 \\ -6 \end{bmatrix}$$

$W = \text{np.array}(\begin{bmatrix} 1 & -3 & 5 \\ 2 & 4 & -6 \end{bmatrix})$ 2 by 3

$$b_1^{[l]} = -1 \quad b_2^{[l]} = 1 \quad b_3^{[l]} = 2$$

$b = \text{np.array}([-1, 1, 2])$

$$\bar{a}^{[0]} = \bar{x}$$

$a_{\text{in}} = \text{np.array}([-2, 4])$

```
def dense(a_in, W, b):  
    3 units = W.shape[1]  [0,0,0]  
    a_out = np.zeros(units)  
    for j in range(units): 0,1,2  
        w = W[:,j]  
        z = np.dot(w, a_in) + b[j]  
        a_out[j] = g(z)  
    return a_out
```

Define the number
Of neurons

Initialize a_out with 0

Take each column in W

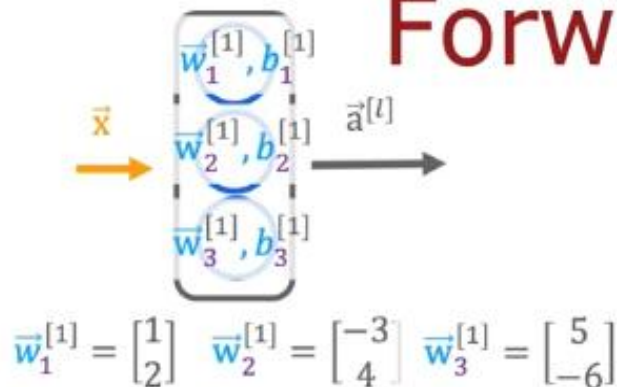
Calculate Z

Note: g() is defined outside of dense().
(see optional lab for details)

Apply the activation function

Output Activation matrix

Forward prop in NumPy



```
W = np.array([  
    [1, -3, 5]  
    [2, 4, -6]]) 2 by 3
```

$$b_1^{[l]} = -1 \quad b_2^{[l]} = 1 \quad b_3^{[l]} = 2$$

```
b = np.array([-1, 1, 2])
```

$$\vec{a}^{[0]} = \vec{x}$$

```
a_in = np.array([-2, 4])
```

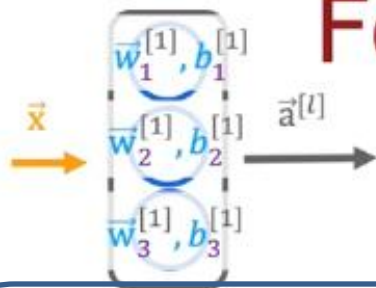
Define the sequential function

The output of the previous layer
Is the input of the next layer

F_x in the output of the
Neural Network

```
def sequential(x):  
    a1 = dense(x, w1, b1)  
    a2 = dense(a1, w2, b2)  
    a3 = dense(a2, w3, b3)  
    a4 = dense(a3, w4, b4)  
    f_x = a4  
    return f_x
```

Forward prop in NumPy



$$\vec{w}_1^{[1]} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad \vec{w}_2^{[1]} = \begin{bmatrix} -3 \\ 4 \end{bmatrix} \quad \vec{w}_3^{[1]} = \begin{bmatrix} 5 \\ -6 \end{bmatrix}$$

```
W = np.array([
    [1, -3, 5],
    [2, 4, -6]]) 2 by 3
```

$$b_1^{[l]} = -1 \quad b_2^{[l]} = 1 \quad b_3^{[l]} = 2$$

```
b = np.array([-1, 1, 2])
```

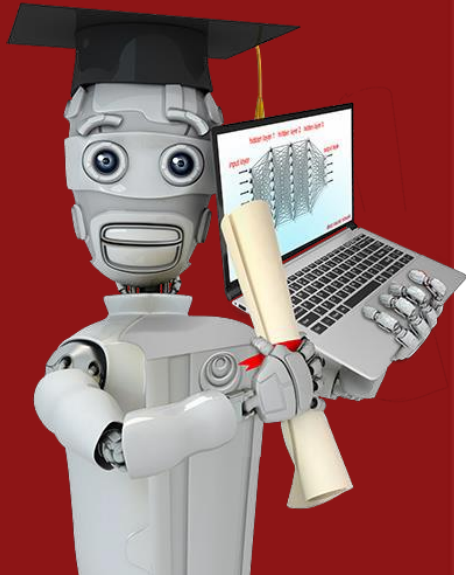
$$\vec{a}^{[0]} = \vec{x}$$

```
a_in = np.array([-2, 4])
```

```
def dense(a_in, W, b):
    3 units = W.shape[1] [0,0,0]
    a_out = np.zeros(units)
    for j in range(units): 0, 1, 2
        w = W[:, j]
        z = np.dot(w, a_in) + b[j]
        a_out[j] = g(z)
    return a_out
```

Note: $g()$ is defined outside of $\text{dense}()$.
(see optional lab for details)

```
def sequential(x):
    a1 = dense(x, W1, b1)
    a2 = dense(a1, W2, b2)
    a3 = dense(a2, W3, b3)
    a4 = dense(a3, W4, b4)
    f_x = a4
    return f_x
```



Speculations on artificial general intelligence (AGI)

Is there a path to AGI?

AI

```
graph TD; AI[AI] --- ANI[ANI]; AI --- AGI[AGI];
```

ANI

(artificial narrow intelligence)

E.g., smart speaker,
self-driving car, web search,
AI in farming and factories

AGI

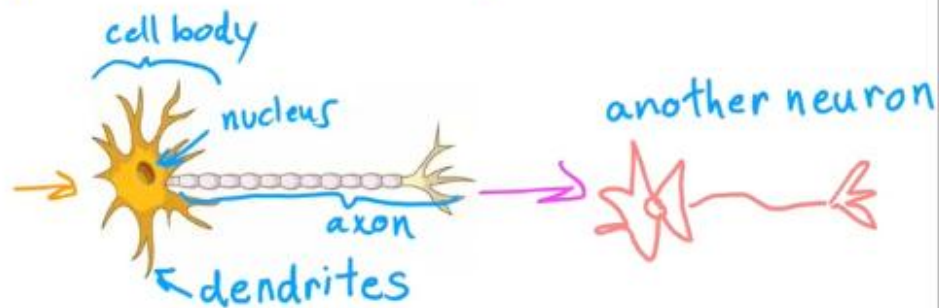
(artificial general intelligence)

Do anything a human can do

Biological neuron

inputs

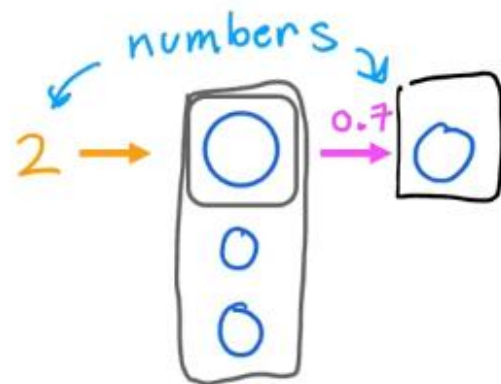
outputs



Simplified mathematical model of a neuron

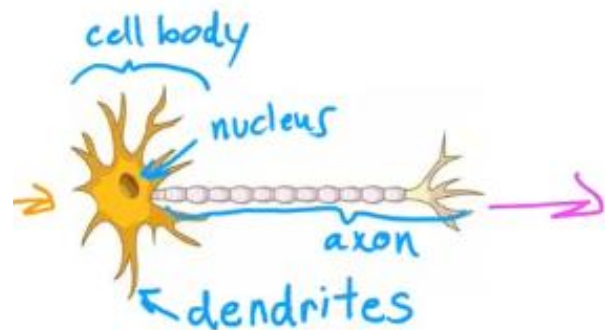
inputs

outputs

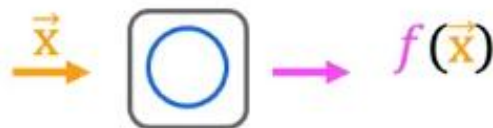


Neural network and the brain

Can we mimic the human brain?

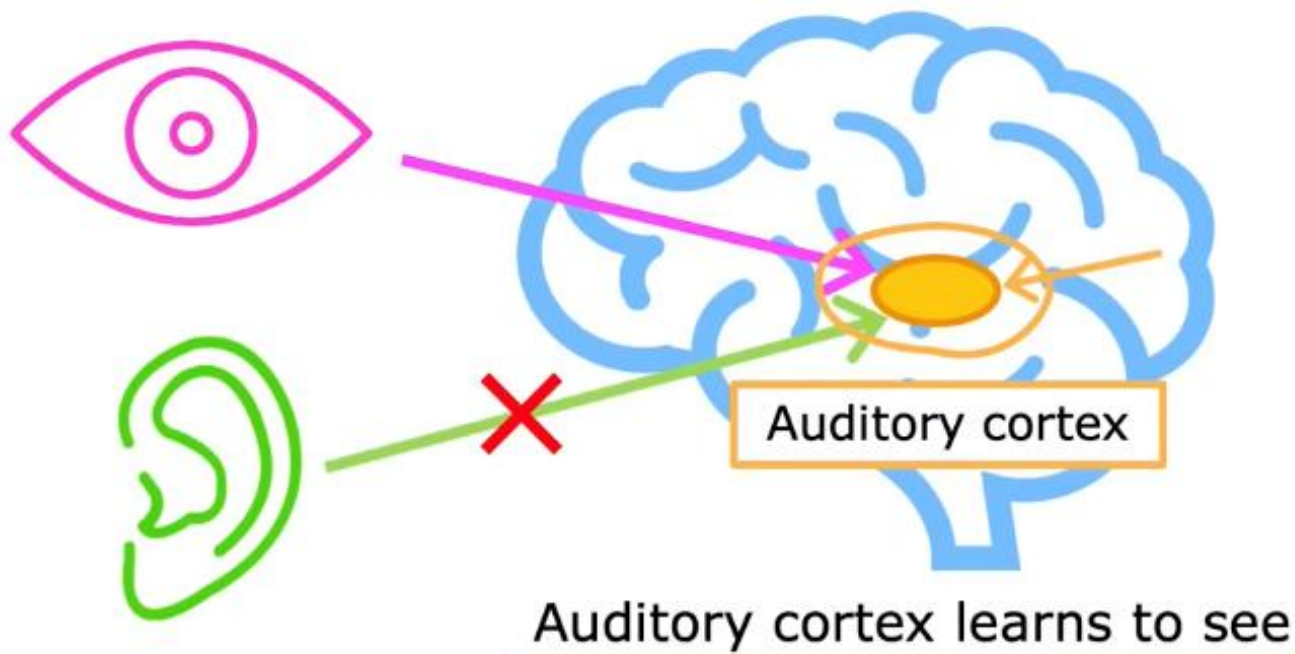


vs

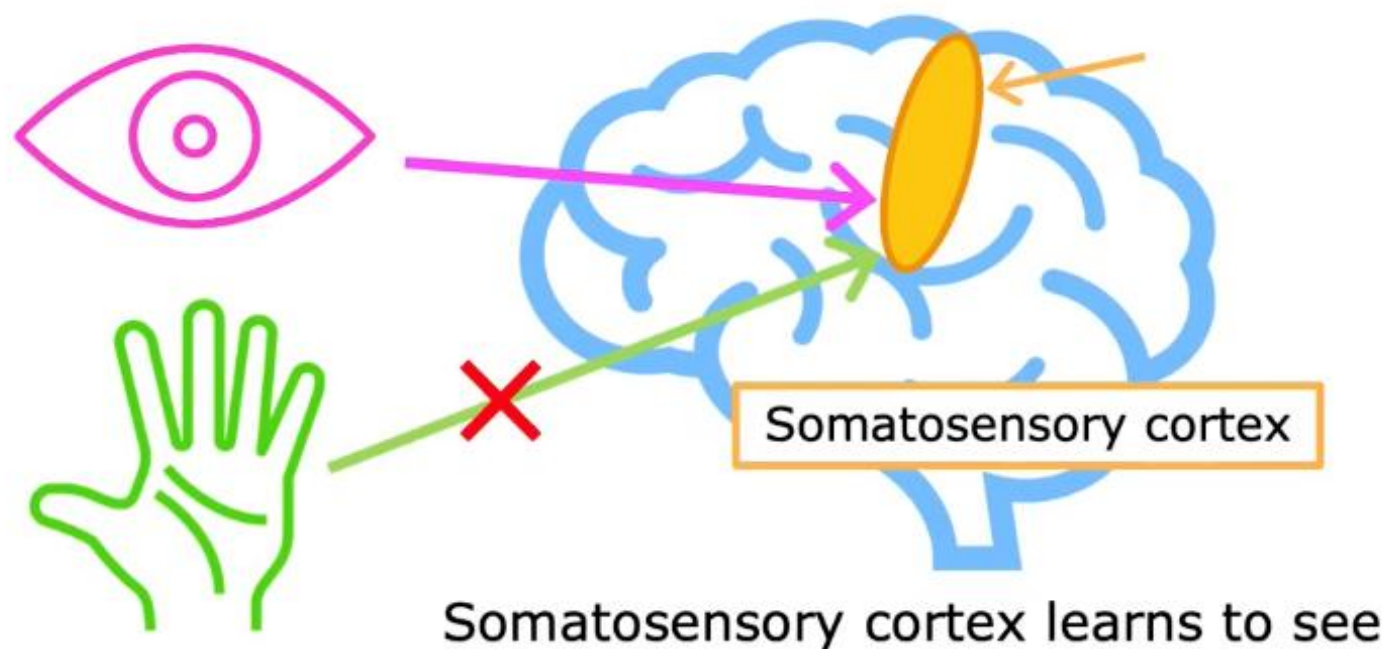


We have (almost) no idea how the brain works

The "one learning algorithm" hypothesis



The "one learning algorithm" hypothesis



Sensor representations in the brain



Seeing with your tongue



Human echolocation (sonar)

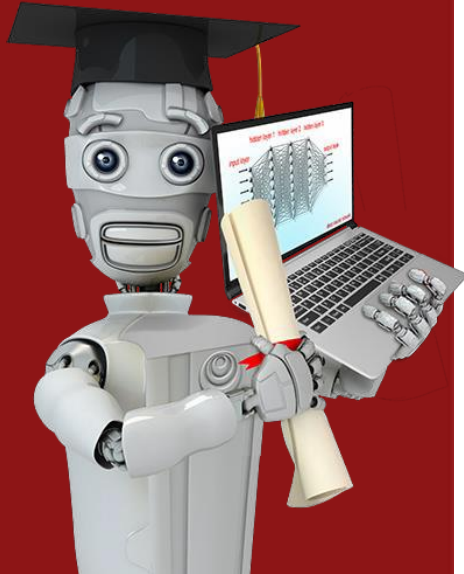


Haptic belt: Direction sense



Implanting a 3rd eye

rainPort; Welsh & Blasch, 1997; Nagel et al., 2005; Constantine-Paton & Law, 2009]



Vectorization

How neural networks are
implemented efficiently

For loops vs. vectorization

```
x = np.array([200, 17])
W = np.array([[1, -3, 5],
              [-2, 4, -6]])
b = np.array([-1, 1, 2])
```

```
def dense(a_in,W,b):
    units = W.shape[1]
    a_out = np.zeros(units)
    for j in range(units):
        w = W[:,j]
        z = np.dot(w, a_in) + b[j]
        a_out[j] = g(z)
    return a_out
```

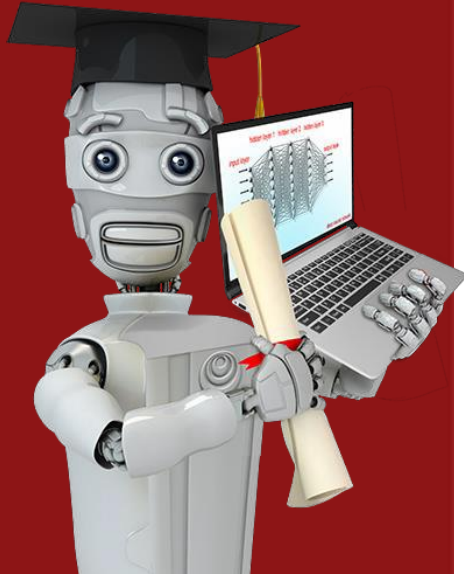
[1,0,1]

```
X = np.array([[200, 17]])
W = np.array([[1, -3, 5],
              [-2, 4, -6]])
B = np.array([-1, 1, 2])
```

```
def dense(A_in,W,B):
    Z = np.matmul(A_in,W) + B
    A_out = g(Z)
    return A_out
```

vectorized

matrix multiplication



Vectorization

Matrix Multiplication

Dot products

example

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix} \cdot \begin{bmatrix} 3 \\ 4 \end{bmatrix}$$

$$z = (1 \times 3) + (2 \times 4)$$

3 + 8
11

in general

$$\begin{bmatrix} \uparrow \\ \vec{a} \\ \downarrow \end{bmatrix} \cdot \begin{bmatrix} \uparrow \\ \vec{w} \\ \downarrow \end{bmatrix}$$

$$z = \vec{a} \cdot \vec{w}$$

transpose

$$\vec{a} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$\vec{a}^T = [1 \quad 2]$$

vector vector multiplication

$$\begin{bmatrix} \leftarrow \vec{a}^T \rightarrow \end{bmatrix} \begin{bmatrix} \uparrow \\ \vec{w} \\ \downarrow \end{bmatrix}$$

1×2 2×1

$$z = \vec{a}^T \vec{w}$$

useful for understanding
matrix multiplication

equivalent

Vector matrix multiplication

$$\vec{a} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$\vec{a}^T = \begin{bmatrix} 1 & 2 \end{bmatrix}$$

$$\mathbf{w} = \begin{bmatrix} 3 \\ 4 \end{bmatrix} \begin{bmatrix} 5 \\ 6 \end{bmatrix}$$

$$\mathbf{z} = \vec{a}^T \mathbf{w} \quad \left[\leftarrow \vec{a}^T \rightarrow \right] \begin{bmatrix} \uparrow \\ \vec{w}_1 \\ \downarrow \end{bmatrix} \begin{bmatrix} \uparrow \\ \vec{w}_2 \\ \downarrow \end{bmatrix}$$

1 by 2

$$\mathbf{z} = [\vec{a}^T \vec{w}_1 \quad \vec{a}^T \vec{w}_2]$$

$$\begin{aligned} (1 * 3) + (2 * 4) \\ 3 + 8 \\ 11 \end{aligned}$$

$$\begin{aligned} (1 * 5) + (2 * 6) \\ 5 + 12 \\ 17 \end{aligned}$$

$$\mathbf{z} = [11 \quad 17]$$

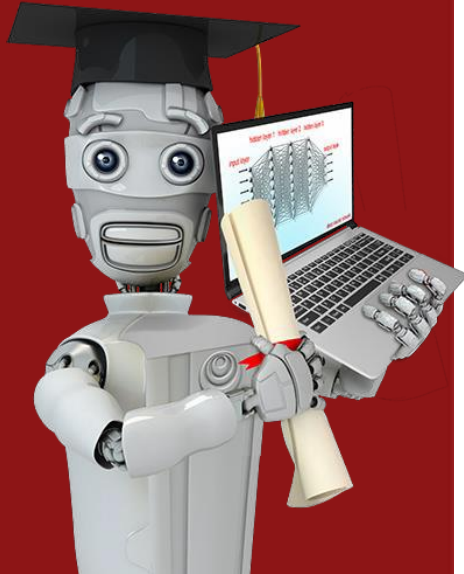
matrix matrix multiplication

$$A = \begin{bmatrix} 1 & -1 \\ 2 & -2 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \end{bmatrix} \quad W = \begin{bmatrix} 3 & 5 \\ 4 & 6 \end{bmatrix} \quad Z = A^T W = \begin{bmatrix} \leftarrow \vec{a}_1^T \rightarrow & \leftarrow \vec{a}_2^T \rightarrow \end{bmatrix} \begin{bmatrix} \uparrow \vec{w}_1 \downarrow & \uparrow \vec{w}_2 \downarrow \end{bmatrix}$$

rows columns

$$\begin{array}{l} \text{row 1 col 1} = \vec{a}_1^T \vec{w}_1 \\ \text{row 2 col 1} = \vec{a}_2^T \vec{w}_1 \\ \text{row 1 col 2} = \vec{a}_1^T \vec{w}_2 \\ \text{row 2 col 2} = \vec{a}_2^T \vec{w}_2 \end{array}$$
$$\begin{array}{l} (-1 \times 3) + (-2 \times 4) \\ -3 + -8 \\ -11 \end{array} \quad \begin{array}{l} (-1 \times 5) + (-2 \times 6) \\ -5 + -12 \\ -17 \end{array}$$
$$= \begin{bmatrix} 11 & 17 \\ -11 & -17 \end{bmatrix}$$

general rules for
matrix multiplication
↪ next video!



Vectorization

Matrix Multiplication rules

Matrix multiplication rules

$$A = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \quad \vec{a}_1^T \vec{a}_2^T \vec{a}_3^T \quad W = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix} \quad \vec{w}_1 \vec{w}_2 \vec{w}_3 \vec{w}_4 \quad Z = A^T W = \begin{bmatrix} \end{bmatrix}$$

$$A = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \quad \vec{a}_1^T \vec{a}_2^T \vec{a}_3^T \quad W = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix} \quad \vec{w}_1 \vec{w}_2 \vec{w}_3 \vec{w}_4 \quad Z = A^T W = \begin{bmatrix} & & & \\ & & & \\ & & & \end{bmatrix}$$

Matrix multiplication rules

$$A = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \quad W = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix} \quad Z = A^T W = \begin{bmatrix} \bigcirc & & & \\ & \bigcirc & & \\ & & \bigcirc & \end{bmatrix}$$

3 by 4 matrix

$$\vec{a}_1^T \vec{w}_1 = (1 \times 3) + (2 \times 4) = 11$$

row 3 column 2

$$\vec{a}_3^T \vec{w}_2 = (0.1 \times 5) + (0.2 \times 6) = 1.7$$

0.5 + 1.2

row 2 column 3?

$$\vec{a}_2^T \vec{w}_3 = (-1 \times 7) + (-2 \times 8) = -23$$

-7 + -16

Matrix multiplication rules

$$A = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \quad W = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix} \quad Z = A^T W = \begin{bmatrix} 11 & 17 & 23 & 9 \\ -11 & -17 & -23 & -9 \\ 1.1 & 1.7 & 2.3 & 0.9 \end{bmatrix}$$

$$\vec{a}_1^T \vec{w}_1 = (1 \times 3) + (2 \times 4) = 11$$

3 by 4 matrix

row 3 column 2

$$\vec{a}_3^T \vec{w}_2 = (0.1 \times 5) + (0.2 \times 6) = 1.7$$

0.5 + 1.2

row 2 column 3?

$$\vec{a}_2^T \vec{w}_3 = (-1 \times 7) + (-2 \times 8) = -23$$

-7 + -16

Matrix multiplication rules

$$A = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \quad W = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix} \quad Z = A^T W = \begin{bmatrix} 11 & 17 & 23 & 9 \\ -11 & -17 & -23 & -9 \\ 1.1 & 1.7 & 2.3 & 0.9 \end{bmatrix}$$

3×2 2×4

can only take dot products
of vectors that are same length

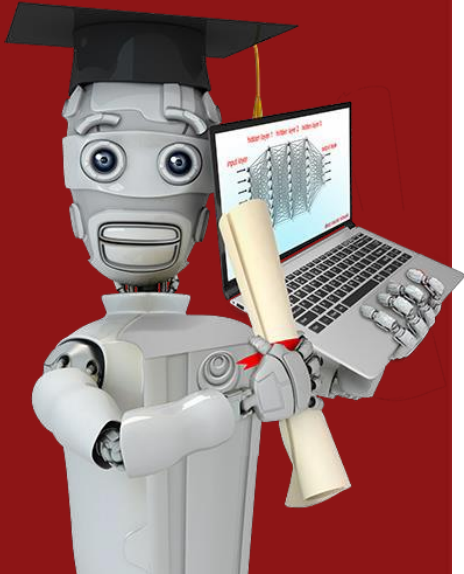
$[0.1 \ 0.2]$

length 2

$\begin{bmatrix} 5 \\ 6 \end{bmatrix}$

length 2

3 by 4 matrix
↳ same # rows as A^T
↳ same # columns as W



Vectorization

Matrix Multiplication Code

Matrix multiplication in NumPy

$$A = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \quad W = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix} \quad Z = A^T W = \begin{bmatrix} 11 & 17 & 23 & 9 \\ -11 & -17 & -23 & -9 \\ 1.1 & 1.7 & 2.3 & 0.9 \end{bmatrix}$$

```
A=np.array([[1,-1,0.1],  
            [2,-2,0.2]])
```

```
W=np.array([[3,5,7,9],  
            [4,6,8,0]])
```

```
Z = np.matmul(AT,W)
```

or
Z = AT @ W

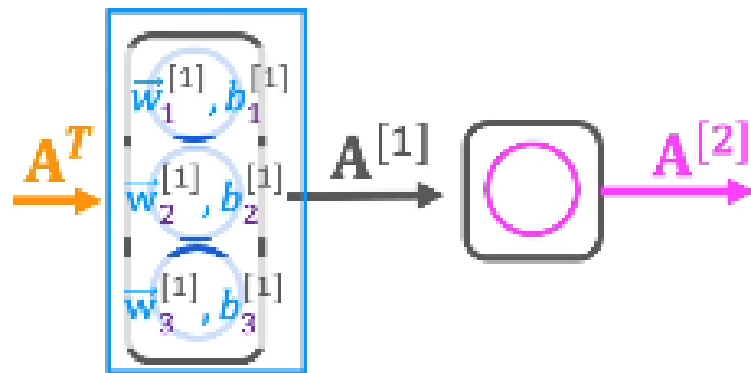
```
AT=np.array([[1,2],  
             [-1,-2],  
             [0.1,0.2]])
```

```
AT=A.T  
    ↳ transpose
```

result

```
[[11,17,23,9],  
 [-11,-17,-23,-9],  
 [1.1,1.7,2.3,0.9]]
```

Dense layer vectorized



$$A^T = \begin{bmatrix} 200 & 17 \end{bmatrix}$$

$$W = \begin{bmatrix} 1 & -3 & 5 \\ -2 & 4 & -6 \end{bmatrix}$$

$$B = \begin{bmatrix} -1 & 1 & 2 \end{bmatrix}$$

$$Z = A^T W + B$$

$$\begin{bmatrix} 165 & -531 & 900 \end{bmatrix}$$

$$z_1^{[1]} \quad z_2^{[1]} \quad z_3^{[1]}$$

$$A = g(Z)$$

$$\begin{bmatrix} 1 & 0 & 1 \end{bmatrix}$$

```

A
AT = np.array([[200, 17]])
W = np.array([[1, -3, 5],
               [-2, 4, -6]])
b = np.array([-1, 1, 2])
a_in
def dense(AT,W,b):
    z = np.matmul(AT,W) + b
    a_out = g(z)
    return a_out
a_in
[[1,0,1]]
    
```